VLSI Design

Simon Dorrer
Johannes Kepler University
simon.dorrer@jku.at

2026-10-29

Table of contents

1 Introduction 2
1.1 Preamble on the Use of AI 3
1.2 Learning Objectives 3
1.3 Prerequisites 4
1.4 Course Content 4
1.5 Grading 5
1.6 Why ASICs? 5
1.7 A Taxonomy of ASICs 5
1.8 The Digital Design Flow 6
1.9 The IIC-OSIC-TOOLS Container 8
1.10 The AMS Chip Template 9
1.10.1 Flow Overview 9
1.10.2 Folder Structure and Makefiles 10
1.11 Agentic AI Chip Design 10
1.12 Example: The TinyWhisper SoC 11
1.12.1 Weak Signal Propagation Reporter 11
1.12.2 Chip Overview 12
1.13 Tools Installation 13
1.13.1 Local Installation 13
1.13.2 Containerized On-Demand Workspace System (COWS) 14
2 Hardware Description with SystemVerilog 14
2.1 Learning Objectives 15
2.2 Recap: Microcontroller Software versus Hardware Description 15
2.2.1 Computing in Time and in Space 17
2.2.2 Synchronous Sequential Design 18
2.3 The HDL Landscape 21
2.3.1 Why SystemVerilog in This Course 26
2.3.2 VHDL versus SystemVerilog 26
2.4 Cheat-Sheets, Guidelines, and Further Reading 27
2.5 SystemVerilog for VHDL Designers 27
2.5.1 Modules, Ports, and Parameters 29
2.5.2 Data Types and Literals 30
2.5.3 Operators and Expression Widths 31
2.5.4 Concurrent Assignments 33

2.5.5 Processes: `always_comb`, `always_ff`, and `always_latch` 35
2.5.6 Registers and Resets 36
2.5.7 The Two-Process Style 37
2.5.8 `if`, `case`, and Latches 41
2.5.9 Finite State Machines 45
2.5.10 Hierarchy and Generate 53
2.5.11 Packages, Typedefs, and Functions 54
2.5.12 What Is Synthesizable 56
2.5.13 Verilog versus SystemVerilog 56
2.6 Testbenches 62
2.6.1 A SystemVerilog Testbench 62
2.6.2 A cocotb Testbench 64
2.6.3 SystemVerilog or cocotb? 66
2.7 Linting, Simulation, and Synthesis in the IIC-OSIC-TOOLS Container 66
2.7.1 Setting Up a Working Folder 66
2.7.2 Linting with Verilator 67
2.7.3 Simulating a SystemVerilog Testbench 68
2.7.4 Running a cocotb Testbench 68
2.7.5 Viewing Waveforms 68
2.7.6 A Quick Synthesis Check with Yosys 69
2.8 Hands-On: The Counter of the AMS Chip Template 82
2.9 HDL Coding Guidelines 87
2.10 Summary 89
3 Modeling Guidelines for Synthesizable RTL 90
4 Functional Verification with cocotb 90
5 Sequential Elements and Static Timing Analysis 90
6 The Synthesis Library 90
7 Synthesis Optimizations 90
8 Design for Test (DFT) 91
9 Clock Domains and Metastability 91
10 The RTL-to-GDSII Flow with LibreLane 91
11 Capstone Project: A Small RISC-V SoC 91
12 Appendix: SystemVerilog Cheat-Sheet 91
13 Appendix: Tooling Cheat-Sheet 91
14 Appendix: Sample Exam Questions 91
Bibliography 92

1 Introduction

This is the material for an introductory-level course on digital **very-large-scale integration (VLSI)** design, held at JKU under course number 336.051 ("KV VLSI Design").

The course teaches the design of **digital application-specific integrated circuits (ASICs)** with a modern, fully **open-source** RTL-to-GDSII flow. We model hardware in **SystemVerilog**, verify it with **cocotb** (Python), inspect waveforms in **GTKWave**/**Surfer**, and harden, assemble, and tape out a complete chip, including the padframe, with **LibreLane** on the **IHP**

SG13CMOS5L 130 nm open PDK. In the course project, you take your own **RISC-V SoC** all the way to a tapeout-ready chip.

> ! Important
>
> All course material (the source of this document, the SystemVerilog code, the cocotb testbenches, the LibreLane configurations, and the Python sources for all figures) is made publicly available on GitHub (follow this link) and shared under the Apache-2.0 license.
>
> Please feel free to submit pull requests on GitHub to fix errors and omissions!

> ! Important
>
> Please use the IIC-OSIC-TOOLS image with tag `2026.09` or later. It contains all tools and the PDK used in this course.

1.1 Preamble on the Use of AI

Artificial intelligence offers impressive possibilities today: it can explain how things are connected, provide examples, help with debugging, or point out different ways to solve a problem. Precisely for this reason, however, there is a risk of relying too heavily on such tools while learning, and thereby not practicing enough of exactly those skills that this course is meant to build.

In the written exam, you will have to solve problems without any technical aids. There, the only thing that counts is which concepts, methods, and skills you have mastered yourself. Likewise, in the oral presentation of your project, you have to be able to explain what you did and why you did it.

If you want to complete a long-distance run in a few months, you have to build up your own endurance. You could, of course, cover your weekly training route faster on an e-scooter, but that does not train your ability to run the distance yourself. The same holds for your studies: if AI regularly does the actual thinking for you, you will not sufficiently build up the skills you need.

So feel free to use AI as a learning partner, but not as a substitute for your own thinking. Always try to understand and solve a problem on your own first. Afterwards, use AI, for example, to check your solution, to learn about an alternative approach, or to have a difficult topic explained to you. The goal of this course is not to arrive at a correct answer as quickly as possible, but to develop the competence to get there on your own.

In the end, you are here to learn something, so do not cheat yourself! This matters beyond the course, too: as AI agents take over more and more routine design work (see Section 1.11), engineers who know how to prompt these tools and, above all, how to critically review their results will always be needed. And you can only review what you understand yourself.

1.2 Learning Objectives

After completing this course, you will be able to:

- work with the **IIC-OSIC-TOOLS** container and manage your designs with **Git** and **GitHub**;

- read and write synthesizable **SystemVerilog** RTL for synchronous digital designs, and translate idiomatic VHDL into idiomatic SystemVerilog;
- apply **modeling guidelines** that lead to clean, fast, and testable synthesis results (avoiding latches, inferring proper registers, structuring arithmetic);
- verify RTL with **self-checking cocotb testbenches** and reason about coverage and corner cases;
- understand and explain the **RTL-to-GDSII flow** (synthesis, floorplanning, placement, clock tree synthesis, routing, signoff) and drive it with **LibreLane**;
- analyze timing using the concepts of **setup/hold, propagation delay, clock skew, and static timing analysis (STA)**;
- explain **design for test (DFT)**, the stuck-at fault model, scan chains, and test coverage;
- handle **multiple clock domains** safely with synchronizers and reason about metastability;
- integrate a digital macro into a **padframe** and assemble a complete, **tapeout-ready ASIC** with open-source tools;
- design, verify, harden, and document a **RISC-V SoC with your own extension** as a capstone project;
- optionally, use **LLM agents** efficiently, and critically, in a chip design workflow.

1.3 Prerequisites

The course is an elective for students of the Bachelor's program in Computer Science and of the Master's program in Electronics and Information Technology (ELIT). To follow it, you should bring:

- a basic understanding of CMOS transistors;
- a good understanding of digital logic (gates, flip-flops, etc.);
- basic experience with a hardware description language (e.g., VHDL);
- your own laptop with at least 20 GB of free disk space for VS Code, Git, and the IIC-OSIC-TOOLS (see Section 1.13).

The following JKU courses, whose material is also openly available, complement this course well but are not required:

- **KV Analog Circuit Design**: https://github.com/iic-jku/analog-circuit-design
- **KV Design of Complex Integrated Circuits**: https://github.com/iic-jku/design-complex-ic

1.4 Course Content

After this introduction, the course covers the following topics:

1. hardware description with SystemVerilog, and modeling guidelines for synthesizable RTL (Section 2, Section 3);
2. functional verification with cocotb (Section 4);
3. sequential elements and static timing analysis (Section 5);
4. the synthesis library and synthesis optimizations (Section 6, Section 7);
5. design for test and clock domain crossing (Section 8, Section 9);
6. the RTL-to-GDSII flow with LibreLane, chip assembly, and padframe (Section 10);
7. the project: a tapeout-ready RISC-V SoC (Section 11).

The appendices provide a SystemVerilog cheat-sheet (Section 12), a tooling cheat-sheet (Section 13), and sample exam questions (Section 14).

> **Further Reading**
>
> These notes are self-contained but kept concise. For depth we recommend:
>
> - [1] for digital design and RISC-V computer architecture.
> - [2] for CMOS VLSI design from circuits to systems.
> - [3] for clock-domain-crossing techniques in SystemVerilog.

1.5 Grading

The grade consists of two equally weighted parts:

- **Part 1 — Theory (50%)**: The theoretical lectures are mixed with small examples. This part is assessed by a **written exam before Christmas**; the exact date is announced in the lecture.
- **Part 2 — Project (50%)**: You implement a complete, **tapeout-ready RISC-V CPU** plus your **own extension** (analog or digital) and present your work in an **oral presentation** (Section 11).

> **Tip**
>
> The best project might actually be taped out! Moreover, the project can be extended into a seminar work (+3 ECTS) or a thesis.

1.6 Why ASICs?

Application-specific integrated circuits are designed when standard, off-the-shelf components cannot meet the requirements. Typical motivations are:

- **Specification**: performance, power, or size targets unreachable with standard parts.
- **Integration**: combining digital logic, analog circuits, and on-chip memory into a single mixed-signal chip.
- **Economics**: at high volume the per-unit cost drops dramatically, and integrating more functionality into one chip reduces system cost.

Historically, the cost *per transistor* has fallen by orders of magnitude as integration density doubled roughly every 18–24 months (**Moore's law**). What was once only affordable to large companies is, with open-source PDKs and tools, now accessible to students and researchers.

1.7 A Taxonomy of ASICs

ASIC implementation styles trade off non-recurring engineering (NRE) cost, unit cost, performance, and design effort:

Table 1: ASIC implementation styles

Style	Masks	Effort	Typical use
PLD / FPGA	none (field-programmable)	low	prototypes, low volume
Gate array / sea-of-gates	only wiring layers	medium	medium volume
Standard-cell IC (CBIC)	all layers	medium–high	the workhorse of digital design
Full-custom IC	all layers, hand-crafted	very high	highest performance / lowest unit cost

This course focuses on the **standard-cell** style: we describe behavior in SystemVerilog and let the synthesis tool map it onto a library of pre-characterized standard cells from the SG13CMOS5L PDK.

1.8 The Digital Design Flow

The core of this course is the digital design flow that turns a hardware description into a manufacturable layout. Since this layout is traditionally handed over to the foundry in the GDSII file format, it is called the **RTL-to-GDSII** (or simply RTL-to-GDS) flow. Figure 1 shows its main steps.

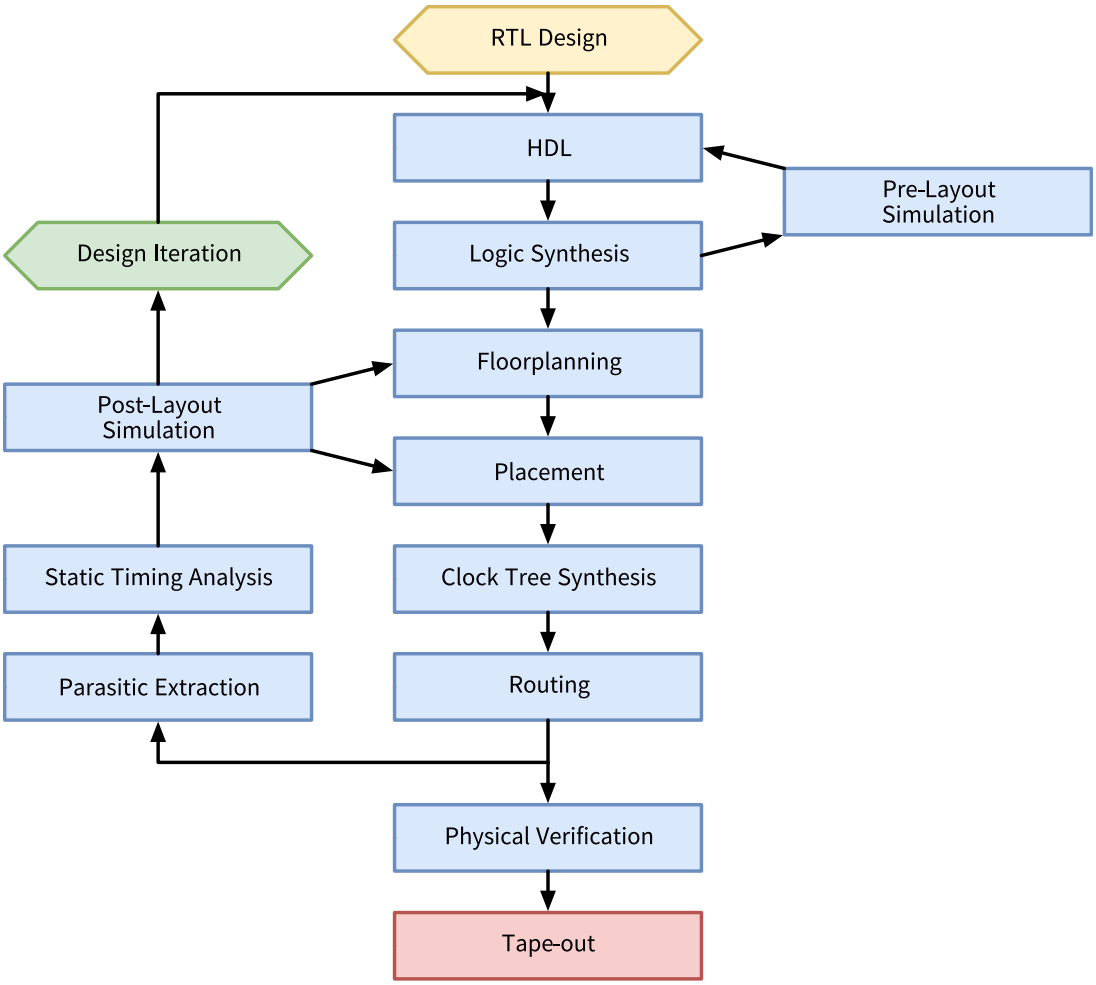


Figure 1: The RTL-to-GDSII design flow. The synthesized gate-level netlist is checked by pre-layout simulation. After routing, parasitic extraction, static timing analysis, and post-layout simulation feed back to floorplanning and placement or, if necessary, to the RTL (design iteration). Only a design that passes physical verification is taped out.

The flow starts with the **RTL design**, described in a hardware description language (HDL) such as SystemVerilog. **Logic synthesis** translates the RTL into a gate-level netlist of standard cells of the target technology, guided by constraints such as the clock frequency. Long before any layout exists, the design is simulated extensively: first at RTL, and then as a synthesized netlist (**pre-layout simulation**). Every bug found here leads back to the HDL.

The physical implementation then turns the netlist into a layout. **Floorplanning** defines the size of the die and the core, the placement of I/Os and macros, and the power grid. **Placement** positions the standard cells, **clock tree synthesis (CTS)** builds a balanced clock distribution network, and **routing** connects all cells with metal wires.

Only after routing are the resistances and capacitances of the wires known. **Parasitic extraction** annotates them to the netlist, on which **static timing analysis (STA)** and **post-layout simulation** check whether the design still meets its timing and functional specification. Violations are fixed by revisiting floorplanning and placement or, if this is not enough, by changing the RTL itself (**design iteration**), e.g., by pipelining a critical path. Finally, **physical**

verification checks the layout against the design rules of the foundry (design rule check, DRC) and against the netlist (layout versus schematic, LVS). Only a design that passes all checks goes to **tape-out**, i.e., is sent to the foundry for fabrication.

Besides these steps, **design for test** inserts scan chains so that every fabricated chip can be tested (Section 8), and designs with several clocks need safe **clock domain crossings** (Section 9). For a complete chip, the hardened digital macro is finally integrated into a **padframe** and assembled into the chip top level (Section 10). Table 2 lists the open-source tools used in this course for each step and where the step is covered.

Table 2: Steps of the RTL-to-GDSII flow, the open-source tools used for them in this course, and where they are covered. LibreLane orchestrates all steps from synthesis to GDSII. The PDK is IHP SG13CMOS5L (130 nm).

Flow step	Open-source tool	Covered in
HDL	SystemVerilog	Section 2, Section 3
RTL, pre- and post-layout simulation	cocotb with Icarus Verilog or Verilator; GTKWave, Surfer	Section 4
Logic synthesis	Yosys	Section 6, Section 7
Floorplanning, placement, CTS, routing	OpenROAD	Section 10
Parasitic extraction	OpenRCX	Section 10
Static timing analysis	OpenSTA	Section 5, Section 10
Physical verification	Magic, KLayout (DRC); Netgen, KLayout (LVS)	Section 10
Chip assembly and padframe	LibreLane with the AMS chip template	Section 10, Section 11

1.9 The IIC-OSIC-TOOLS Container

The legacy version of this course relied on commercial tools (Vivado, ModelSim, Synopsys Design Compiler, Cadence Innovus, TetraMAX, PrimeTime). This edition is built entirely on open-source tools, all pre-installed in the **IIC-OSIC-TOOLS** container [4]. This all-in-one Docker/Podman image for analog and digital IC design is curated at JKU and runs natively on `x86_64` and `arm64` machines (e.g., Apple Silicon). Since everyone uses the identical container, results are reproducible, and "works on my machine" problems disappear.

As Table 3 shows, the container covers much more than this course needs: full-custom (analog) design with schematic entry, analog simulation, layout, and electromagnetic (EM) simulation, semi-custom (digital) design with simulation, synthesis, and complete RTL-to-GDSII flows, as well as several open-source PDKs. In this course, we mainly use the digital tools (Table 2) with the IHP SG13CMOS5L PDK, and the analog tools appear in the AMS chip template (Section 1.10).

Table 3: A selection of the tools and PDKs in the IIC-OSIC-TOOLS container

Category	Tools and PDKs
Schematic entry	Xschem, Qucs-S
Analog simulation	ngspice, Xyce, VACASK, CACE
Layout editing	Magic, KLayout, GDSFactory
Digital simulation and waveforms	Icarus Verilog, Verilator, cocotb, GTKWave, Surfer
Logic synthesis	Yosys (with the slang and GHDL front ends), ABC
RTL-to-GDSII flows	LibreLane, OpenROAD Flow Scripts, SiliconCompiler, QFlow
Place and route, STA, extraction	OpenROAD, OpenSTA, OpenRCX
DRC, LVS, and PEX	Magic, KLayout, Netgen, KLayout-PEX
EM simulation	Palace, openEMS
PDKs	sky130A, gf180mcuD, ihp-sg13g2, ihp-sg13cmos5l

1.10 The AMS Chip Template

Hardening a single digital block is only part of the way to a chip. A tapeout-ready chip additionally needs a padframe and a seal ring, the placement and wiring of all macros on the top level, logo and metal-fill insertion, full-chip verification, and finally a bonding diagram for packaging. To get there without fighting the tools, this course builds on the **ihp-sg13cmos5l-ams-chip-template** [5], an open-source GitHub template repository for analog mixed-signal (AMS) chips developed at JKU:

- It is released under the permissive **Solderpad Hardware License v2.1**.
- Create your own repository with GitHub's *Use this template* button (or simply `git clone` it), enter the IIC-OSIC-TOOLS container, and run `make all`. The result is a **tapeout-ready AMS chip**.
- A step-by-step **tutorial**, written in Quarto, walks through all IC design steps on IHP's 130 nm process, with simple exercises to learn by doing. You start your project with this tutorial (Section 11).
- **Regression tests** via GitHub Actions ensure that the flow keeps working with every new release of the tools.

The template is also available for the ihp-sg13g2, sky130A, and gf180mcuD open PDKs, and several chips have already been taped out with it, including TinyWhisper (Section 1.12).

1.10.1 Flow Overview

The Makefile-driven flow of the template covers:

- simulation, linting, and hardening of the RTL;
- layout checks and reports of the hardened digital macro;
- simulation of all analog testbenches, including process, voltage, and temperature (PVT) corners and Monte Carlo (MC) analysis;

- layout checks of the analog and digital blocks (DRC, LVS, and parasitic extraction, PEX);
- digital-on-top assembly of the chip top level and its layout check (DRC), including
  - seal ring and padframe generation,
  - macro placement and wiring, also with non-default rules (NDRs), and
  - logo and fill insertion;
- full top-level LVS and simulation without black-boxing;
- generation of the bonding diagram.

1.10.2 Folder Structure and Makefiles

The folder structure of the template was discussed with the FOSSi community. Every macro in `macros/` (e.g., the digital `counter` and the analog `inverter` examples) follows the same scheme as the chip top level, with its own `Makefile` and folders such as `rtl/`, `testbenches/`, `schematic/`, `layout/`, `netlist/`, `verification/`, and `final/`. Since a macro can itself contain macros, the structure is recursive. This makes it easy to share macros between projects and to reproduce the results of others.

At the top level, `make all` runs four steps:

1. `make build-all` builds the bondpad, the logos, all enabled macros, and finally the chip top level (LibreLane assembly, reports, GDS, netlists, logo and fill insertion, and renders);
2. `make magic-drc` (or `make klayout-drc`) checks the final layout of the chip;
3. `make sim-all` simulates the chip (RTL and gate-level simulation with cocotb, gate-level simulation with Xschem);
4. `make bondplan` generates the bonding diagram for packaging.

Targets that build an IP block or a macro recursively call the `Makefile` of that block.

1.11 Agentic AI Chip Design

Large language model (LLM) agents are increasingly able to operate a complete chip design environment: they do not only answer questions, but also edit design files, run tools, interpret the results, and iterate. Figure 2 shows such an agentic workflow.

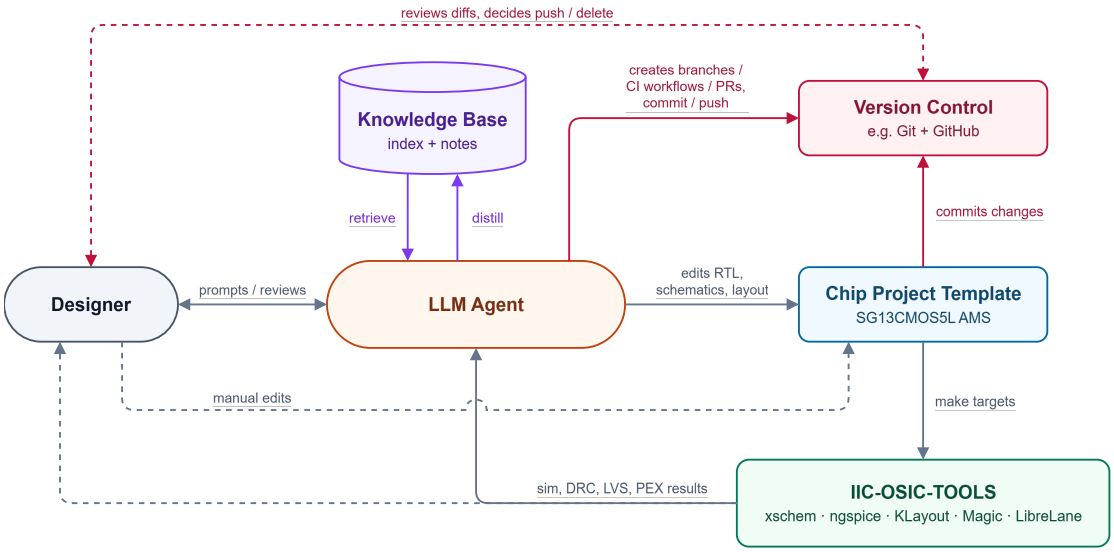


Figure 2: Agentic AI chip design workflow: the designer prompts and reviews an LLM agent, which edits the chip project, runs the tools via make targets, and tracks all changes in version control (shown here for the SG13CMOS5L version of the AMS chip template).

The designer interacts with the LLM agent through prompts and reviews. The agent edits the RTL, schematics, and layout of a chip project based on the AMS chip template (Section 1.10) and runs its make targets, which call the tools of the IIC-OSIC-TOOLS. The results of simulation, DRC, LVS, and PEX flow back to the agent and to the designer. A knowledge base (an index plus notes) lets the agent retrieve relevant information and distill new insights for later sessions. All changes are tracked in version control (e.g., Git and GitHub): the agent creates branches, commits and pushes changes, sets up CI workflows, and opens pull requests, while the designer reviews the diffs and decides whether changes are pushed or discarded. Of course, the designer can still edit the project manually at any time.

Such a workflow is only possible because every step of the flow is scripted, every tool runs in a reproducible container, and every result is machine-readable. Note the role of the designer: they remain responsible for the design and have to judge whether the results of the agent are correct, which brings us back to Section 1.1. Using LLM agents efficiently is an optional learning objective of this course.

1.12 Example: The TinyWhisper SoC

What is possible with open-source tools and the AMS chip template is demonstrated by **TinyWhisper**, an open-source, fully integrated, multi-mode short-wave transmitter for amateur radio, developed in a cooperation between JMU Würzburg and JKU Linz. TinyWhisper combines a RISC-V CPU with peripherals, custom digital signal generation, and an analog front-end on one chip, so it is a good preview of what you will build in the course project. The block diagram and all design files can be found in the repository, and the full documentation is available online.

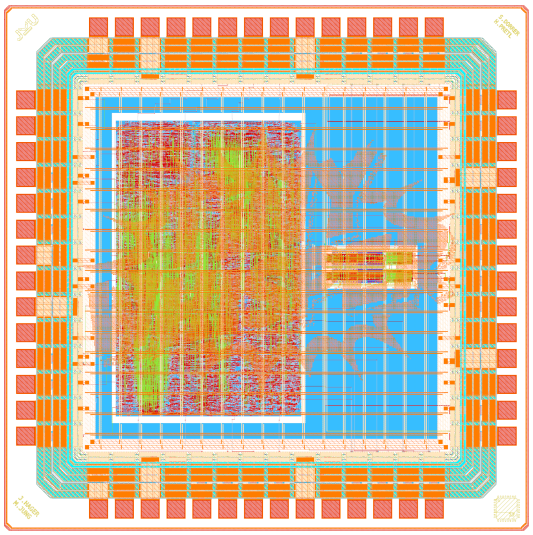
1.12.1 Weak Signal Propagation Reporter

TinyWhisper targets the **Weak Signal Propagation Reporter (WSPR)** protocol, a popular digital mode in amateur (ham) radio for propagation studies in the 80 m to 10 m bands. A WSPR transmitter sends a short message (callsign, location, and transmit power) using 4-

FSK modulation with a tone spacing of $\Delta f = 1.46\,\text{Hz}$ and a symbol duration of $T = 0.683\,\text{s}$. Receivers can decode signals down to $-31\,\text{dB}$ below the noise floor in a 2500 Hz bandwidth and report everything they receive to a central database via the Internet (WSPRnet map). On TinyWhisper, the modem part (coding, framing, and symbol mapping) is implemented in firmware written in C, which runs on the on-chip RISC-V CPU.

1.12.2 Chip Overview

Table 4 summarizes the key parameters of the chip, and Figure 3 shows renders of its layout. The custom logo on the topmost metal layer (Figure 3b) was generated with the tool ArtistIC of the PULP platform. TinyWhisper was taped out in the IHP open MPW run in March 2026 and, at the time of writing (October 2026), is still in fabrication.



(a) Final layout with the RISC-V CPU on the left and the IQ modulator on the right.



(b) TopMetal2 layer with the logo and metal fill.

Figure 3: Renders of the taped-out TinyWhisper chip (2000 µm × 2000 µm), taken from the TinyWhisper repository.

Table 4: Key parameters of the TinyWhisper chip

Parameter	Value
Technology	IHP SG13CMOS (130 nm CMOS), open PDK
Die area	2000 µm × 2000 µm (4 mm²)
Core area	1270 µm × 1270 µm (1.61 mm²)
Clock frequency	56 MHz
Core / I/O supply voltage	1.5 V / 3.3 V
Pads	56
Package	QFN-48 (8 pads bonded directly to package ground)

The **digital core** is built around an embedded RISC-V CPU implementing `rv32e_zicsr`, i.e., the 32-bit embedded base integer instruction set with 16 registers plus the extension for control and status registers. The CPU is a multi-cycle design without pipelining, running at 56 MHz, and accesses its SPI, I²C, UART, and GPIO peripherals as memory-mapped I/O. The I/

Q baseband signals are generated in hardware: a 30-bit iterative CORDIC computes sine and cosine with a frequency resolution of $\Delta f = f_{\mathrm{clk}}/2^{30} \approx 0.05\,\mathrm{Hz}$, and a single-bit second-order delta-sigma modulator (oversampling ratio up to 256) converts its output into a bitstream for the analog front-end. Both are configurable via software. Furthermore, the digital core divides the clock down to the 25% duty-cycle local oscillator (LO) signals of the mixers, which cover the 80 m, 40 m, 20 m, and 10 m bands.

The RISC-V CPU has already been proven in silicon: it was part of a multi-project student chip (with digital filters, mini games, and more) fabricated in the GlobalFoundries GF180MCU technology in the first wafer.space run (GitHub). On a breadboard, with the firmware supplied by an external microcontroller board and a discrete third-order multiple-feedback low-pass filter, the chip generates a clean baseband signal.

The **analog front-end** is an IQ modulator. Each of its two channels consists of a third-order multiple-feedback (MFB) Butterworth low-pass filter with a cutoff frequency of 400 kHz, built around a Barthélemy/Manfredini inverter-based operational transconductance amplifier (OTA), followed by a passive CMOS voltage-mode mixer with simple transmission-gate (TG) switches and a single-to-differential conversion circuit. Remarkably, the front-end consists only of inverter- and TG-based circuits. As an IQ transmitter, it supports not only WSPR but also other modes such as FT8, SSB, and CW.

Finally, even the **bonding diagram** for packaging is generated automatically: a Python script reads a configuration file with the pinout and placement, extracts the relevant layers (top metal and pads) from the layout, places the chip in the package, determines the pad centers, draws the bond wires, and renders the result as GDS. This is exactly what `make bondplan` of the AMS chip template does (Section 1.10.2).

> "The aha moment of seeing your circuit come to life will stay long after the equations are forgotten." — Peter Kinget

1.13 Tools Installation

You can either install the IIC-OSIC-TOOLS locally on your own laptop or use a workspace on a JKU server via your browser.

1.13.1 Local Installation

1. Install **Visual Studio Code** from https://code.visualstudio.com.
2. Run the interactive **installer**. It sets up Git, Docker (or Podman), WSL2 on Windows or XQuartz on macOS, and clones the IIC-OSIC-TOOLS repository (by default to `~/iic-osic-tools`). Every action asks for your confirmation first.

   On Windows, open PowerShell **as Administrator** and run:

   ```
   irm https://osic.tools/install.ps1 | iex
   ```

   On macOS and Linux, run:

   ```
   curl -fsSL https://osic.tools/install.sh | bash
   ```

3. Reboot your computer. On Windows and macOS, start Docker Desktop once afterwards.
4. Start the container from within the `iic-osic-tools` folder with `.\start_vnc.bat` (Windows) or `./start_vnc.sh` (macOS and Linux), and open http://localhost/?password=abc123 in your browser. The first start downloads the image (about 4 GB), so be patient. With Podman, the default port is 8080 instead of 80; the start script always prints the correct URL. On macOS and Linux, `./start_x.sh` alternatively shows the tools directly on your desktop via X11.
5. Store your designs in `~/eda/designs` on your laptop (`%USERPROFILE%\eda\designs` on Windows), which appears as `/foss/designs` inside the container. Only this folder persists when the container is removed or updated.

> **Tip**
>
> Piping a script from the Internet directly into a shell runs it without review. If you prefer to inspect the installer first, download it, read it, and run it locally, e.g., `curl -fsSLO https://osic.tools/install.sh && less install.sh && bash install.sh`.

1.13.2 Containerized On-Demand Workspace System (COWS)

If you cannot or do not want to install the tools locally, you can use the **Containerized On-Demand Workspace System (COWS)** instead, which provides a ready-to-use design environment in your browser:

1. Open COWS at https://jku128011.riic.uni-linz.ac.at/.
2. Log in with the credentials provided to you and change your password.
3. Create a workspace: go to *Workspaces*, click *Create workspace*, enter a *Workspace name*, and select the *KV VLSI Design 26/27* template.
4. Start the workspace; its *Status* should change to *Running*.
5. Open the workspace by clicking its name.

Enjoy your workspace, and happy designing!

2 Hardware Description with SystemVerilog

Most of you already design digital circuits in VHDL. This chapter is a crash course that maps what you know from VHDL onto synthesizable **SystemVerilog**, the hardware description language (HDL) we use throughout this course. It is not an introduction to digital design: the synchronous, latch-free design style you learned with VHDL carries over unchanged, and only the syntax and a few semantic rules are new. Along the way, you write testbenches in SystemVerilog and in Python with **cocotb**, and you lint, simulate, and synthesize your designs in the IIC-OSIC-TOOLS container.

> **Note**
>
> All code of this chapter (RTL, VHDL references, testbenches, and the solutions of the exercises) is available in the folder `content/sv_crashcourse/code` of the course repository, with one folder per design. The SystemVerilog code follows the layout of a macro in the

AMS chip template: the RTL in `rtl/` and the testbenches in `testbenches/verilog/` and `testbenches/cocotb/`. The VHDL versions are in `vhdl/`, the plain Verilog original of the priority encoder exercise is in `verilog/`, and the lint exercise keeps its file with deliberate mistakes in `exercise/`. Every file in `rtl/` lints without warnings and synthesizes, and all testbenches pass with the tools of the IIC-OSIC-TOOLS image `2026.10`: Verilator 5.052, Icarus Verilog 14.0, cocotb 2.1.0, GHDL 7.0.0-dev (for the VHDL files), and Yosys 0.70. With another image, the cell counts in the solutions can differ slightly.

2.1 Learning Objectives

After this chapter, you will be able to:

- explain how a hardware description differs from software for a microcontroller
- describe a synchronous sequential circuit as a register plus next-state logic, and explain why the combinational logic has one clock period to compute
- name the main families of HDLs and judge their strengths and weaknesses
- translate VHDL designs written in the two-process style into idiomatic SystemVerilog
- read plain Verilog, e.g., netlists and older IP, and translate Verilog RTL into SystemVerilog
- write synthesizable combinational and sequential logic with `always_comb` and `always_ff`, and avoid latches, races, and width errors
- write self-checking testbenches in SystemVerilog and in Python with cocotb
- lint, simulate, and synthesize a design in the IIC-OSIC-TOOLS container by typing the commands yourself, and explain what the corresponding make targets of the AMS chip template do

2.2 Recap: Microcontroller Software versus Hardware Description

A C program for a microcontroller (MCU) and a SystemVerilog module may look alike: both have variables, operators, `if`, and `for`. Yet they describe fundamentally different things. The C program is a list of instructions that a fixed processor executes one after another. The program counter selects the next instruction, and the time a computation takes follows from the number of executed instructions. A hardware description, in contrast, describes the circuit itself. After synthesis, every line has become gates and flip-flops, and all of them work at the same time, all the time. Time is explicit: the registers update at the clock edge, and the combinational logic between them has one clock period to settle. Table 5 summarizes the differences.

Table 5: Software for a microcontroller versus a hardware description

Aspect	Software for an MCU (e.g., C)	Hardware description (HDL)
What is described	a sequence of instructions for a fixed processor	the structure and behavior of the circuit itself
Execution	one instruction after another, controlled by the program counter	all statements and blocks operate concurrently, all the time
Time	implicit, follows from the number of instructions	explicit, every register updates at the clock edge
A variable is	a memory location or a CPU register	a wire (combinational logic) or a flip-flop, inferred from how it is assigned
A loop is	a repetition in time	a repetition in space: unrolled into parallel hardware, the bound must be a constant
A function call is	a jump to shared code	a separate copy of the hardware for every call
Compilation result	machine code in the flash memory	a netlist of gates and flip-flops, then a layout (GDSII)
Cost	memory and clock cycles	area, power, and maximum clock frequency
Debugging	debugger, `printf`, single steps on the target	simulation and waveforms before the tapeout, little visibility in silicon
Fixing a bug	a firmware update	for an ASIC, a new mask set and fabrication run, which takes months

The most important consequence is a change in thinking: **think hardware first, then write the code.** Before you write a module, sketch its block diagram with the registers and the combinational logic between them (Section 2.2.2). The code then only describes this diagram. Keep in mind that a simulator executes your HDL as an event-driven program on your computer. This is why HDLs also contain constructs such as delays, `$display`, or file access, which only make sense in simulation and cannot be synthesized (Section 2.5.12).

In the course project (Section 11), you need both ways of thinking. You build a RISC-V SoC: the CPU and the blocks around it are hardware that you describe as registers and combinational logic, and the CPU runs firmware that you write as for any microcontroller. Many SoCs today contain such a processor core, because the firmware keeps the digital part flexible: a new feature or a bug fix in the control flow needs a firmware update instead of a new mask set, while dedicated hardware handles the time-critical parts. RISC-V is a popular choice for this core, since its instruction set is open and can be used and extended without license fees.

TinyWhisper (Section 1.12) is an example, with its modem in C firmware on the RISC-V CPU and its IQ modulator in dedicated hardware.

2.2.1 Computing in Time and in Space

The difference between software and hardware becomes obvious with a loop. The following C function counts the ones in a byte (population count):

```c
uint8_t popcount(uint8_t x)
{
  uint8_t count = 0;
  for (int i = 0; i < 8; i++)
    count += (x >> i) & 1;
  return count;
}
```

In the loop body, `(x >> i) & 1` shifts bit `i` of `x` into the least significant position and clears all other bits, so `count` grows by one exactly when bit `i` is set. The same function written in SystemVerilog looks almost identical, but the hardware behind it is not a loop at all:

```systemverilog
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Combinational population count (number of ones in x_i).

`default_nettype none

module popcount (
  input  logic [7:0] x_i,
  output logic [3:0] count_o
);

  // The loop is unrolled at elaboration: eight adders, no clock cycles
  always_comb begin
    count_o = '0;
    for (int i = 0; i < 8; i++) begin
      count_o = count_o + 4'(x_i[i]);
    end
  end

endmodule

`default_nettype wire
```

In the loop body, `x_i[i]` selects bit `i` directly, without a shift and a mask, and the size cast `4'(...)` (Section 2.5.3) extends this 1-bit value explicitly to the 4 bits of `count_o`, which SystemVerilog would otherwise do silently.

Exercise: Loops in Software and in Hardware

1. Roughly how many clock cycles does an MCU need for the C function? How long does the hardware need?
2. What does the synthesis tool build from the `for` loop in `popcount.sv`? Is there a loop counter `i` in the hardware?
3. What changes in both cases when `x` has 64 bits instead of 8?
4. Optional, if you already have the container running: save the listing as `popcount.sv`, synthesize it with `yosys -p "read_verilog -sv popcount.sv; synth -top popcount; stat"`, and look at the cell statistics at the end of the log (Section 2.7.6 explains the command).

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

Solution: Loops in Software and in Hardware

1. Per iteration, the MCU executes a shift, an AND, an addition, the increment of `i`, and a compare-and-branch, so about 40 instructions plus the function call. Depending on the core and the compiler, this takes several tens of clock cycles. The hardware needs no clock cycle at all: the module is purely combinational, and the result is valid after the propagation delay of the logic, usually well within one clock period of the surrounding registers.
2. The loop is unrolled at elaboration: the synthesis tool creates a chain of eight adders (and may restructure them into a tree, as long as the function stays the same). The loop variable `i` exists only at compile time, so there is no counter in the hardware. This is why loop bounds in synthesizable code must be constants.
3. The MCU needs eight times as many iterations, so the time grows. The hardware needs eight times as many adders, and wider ones, so the area grows by more than a factor of eight. The result still needs no clock cycle, but the propagation delay grows as well, although an adder tree grows more slowly than a chain. This is the basic trade-off between computing in time and computing in space. Processors often add dedicated hardware for such functions, e.g., the `cpop` instruction of the RISC-V bit-manipulation extension Zbb.
4. Yosys 0.70 reports 26 generic logic cells (`$_XOR_`, `$_NAND_`, `$_ANDNOT_`, and others) and no flip-flops, confirming that the module is purely combinational.

2.2.2 Synchronous Sequential Design

The popcount module is purely combinational: its output depends only on its current inputs. Most circuits also need memory, e.g., for the state of a finite state machine (FSM) or the value of a counter. Almost all of them are built as **synchronous sequential circuits** with the structure of Figure 4. A **register** of edge-triggered flip-flops stores the current state. The **next-state logic**, a purely combinational circuit, computes the next state from the current state and the inputs. At every rising clock edge, the register loads the next state from its D

input, and the new state appears at its Q output. The outputs of the circuit are derived from the state (Moore FSM) or from the state and the inputs (Mealy FSM).

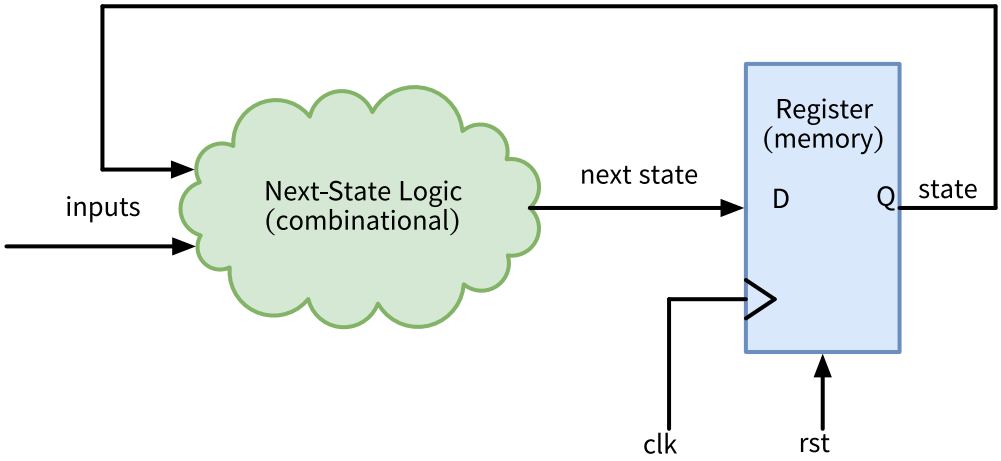


Figure 4: Synchronous sequential design. A register of edge-triggered flip-flops stores the state. The combinational next-state logic computes the next state from the state and the inputs, and the register loads it at every rising clock edge. In the two-process style, the combinational process describes the next-state logic (`next_state_logic` in VHDL, `always_comb` in SystemVerilog), and the register process describes the register (`reg_proc` in VHDL, `always_ff` in SystemVerilog).

Edge-triggered flip-flops are the key to this structure. Their outputs change only at the rising clock edge and stay stable for the rest of the clock period. The next-state logic therefore sees stable inputs for one full clock period, and this is exactly the time it has to compute the next state. The result must have settled before the next rising edge (more precisely, a setup time before it), when the register loads it (Figure 5). The same holds for every path from a register through combinational logic to the next register, so the clock period only has to be longer than the slowest of these paths. Static timing analysis checks exactly this for every path in the design (Section 5). The rule assumes that the inputs also change only after a clock edge, i.e., that they come from registers driven by the same clock. An asynchronous input needs a synchronizer first (Section 9).

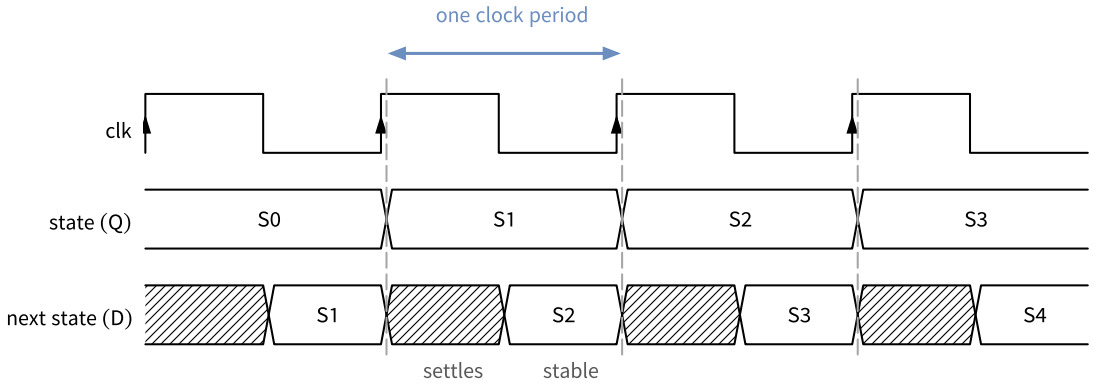


Figure 5: Timing of a synchronous sequential circuit. The state at the Q output of the register changes only at the rising clock edge. The next-state logic then needs some time until the next state at the D input has settled (hatched), and it must be stable before the next rising edge, when the register loads it.

In your VHDL designs, you describe this structure with two processes, a register process `reg_proc` and a combinational process such as `next_state_logic`. As a minimal example, consider a toggle flip-flop, whose state inverts at every rising clock edge at which `toggle_i` is high:

```
-- Register process: only reset and clock in the sensitivity list
reg_proc: process(rst_i, clk_i) is
begin
  if rst_i = '1' then
    state <= '0';
  elsif rising_edge(clk_i) then
    state <= next_state;
  end if;
end process reg_proc;

-- Combinational process: all signals it reads in the sensitivity list
next_state_logic: process(state, toggle_i) is
begin
  -- default assignment
  next_state <= state;

  -- calculate next_state from state and the inputs
  if toggle_i = '1' then
    next_state <= not state;
  end if;
end process next_state_logic;
```

Three rules make this style work:

- The **register process** has only the reset and the clock in its sensitivity list, because the register reacts to nothing else. Its body is always the same: the reset values and, at the rising clock edge, the copies of the `next_` signals. No logic belongs here.

- The sensitivity list of the **combinational process** contains **all signals that are read** in the process. The simulator executes the process only when a signal in this list changes, while the synthesis tool ignores the list (at best with a warning) and builds combinational logic anyway. A missing signal therefore makes the simulation differ from the synthesized hardware. VHDL-2008 avoids this problem with `process(all)`.
- The combinational process starts with a **default assignment**, here `next_state <= state`, followed by all decisions. Every signal is thus assigned on every path, and no latch can arise. Keeping the state is not a latch: the register is the memory, and the next-state logic only feeds the current state back to its D input.

Section 2.5.7 shows the same style and the same three rules in SystemVerilog.

2.3 The HDL Landscape

VHDL and Verilog appeared in the 1980s, and for decades they were practically the only choice. In the last fifteen years, a large number of alternatives has appeared, most of them open source. They fall into four families:

1. **Standardized HDLs.** VHDL (IEEE 1076), Verilog (IEEE 1364), and SystemVerilog (IEEE 1800, which absorbed Verilog in 2009) are read directly by the commercial simulation, synthesis, and verification tools. Among the open-source tools, (System)Verilog has much broader support than VHDL (Table 8).
2. **Embedded domain-specific languages (eDSLs)**, also called *hardware construction languages*. The hardware is described with a library of a general-purpose language such as Scala, Python, or Haskell. Running the program *generates* the hardware as Verilog, so the classes, functions, loops, and type system of the host language are available to build parameterizable generators.
3. **New standalone HDLs** with their own syntax and compiler, designed to avoid the pitfalls of Verilog. They also emit (System)Verilog.
4. **High-level synthesis (HLS) and compiler intermediate representations.** An algorithm written in C/C++ or a dataflow language is scheduled into clock cycles by the tool, or a compiler uses an intermediate language as its hardware back end.

Figure 6 shows how the four families relate.

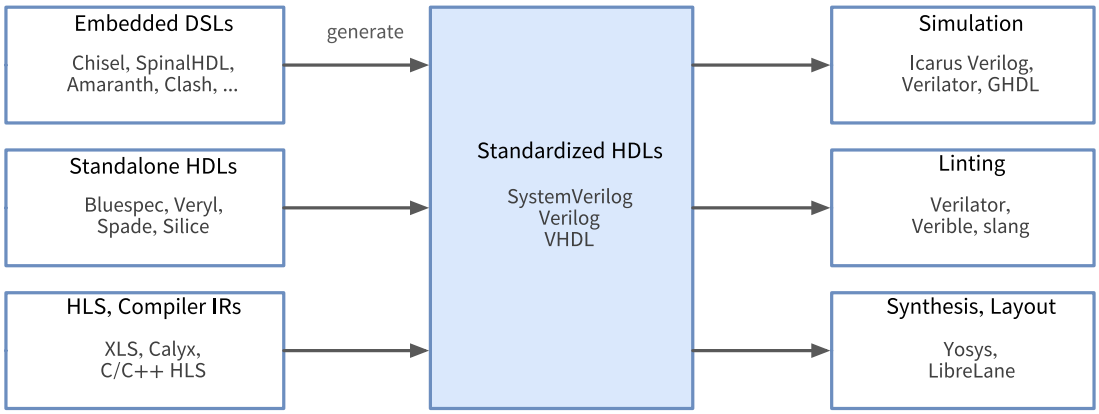


Figure 6: The four families of HDLs. Embedded DSLs, standalone HDLs, and HLS tools generate one of the standardized HDLs, mostly (System)Verilog, which the tools for simulation, linting, synthesis, and layout then read.

Table 6 gives an overview of the languages you are most likely to come across.

Table 6: An overview of hardware description languages. Origins and years are those of the first public release.

Language	Family (host language)	Key idea	Origin	Output
VHDL	standard, IEEE 1076-2019	strongly typed, deterministic signal semantics	US Department of Defense, IEEE standard since 1987	read by commercial tools and GHDL
Verilog	standard, IEEE 1364-2005	weakly typed, compact syntax	Gateway Design Automation, 1984	read by all tools
SystemVerilog	standard, IEEE 1800-2023	Verilog plus design and verification extensions	Accellera (2002), IEEE since 2005	read by all tools
Chisel	eDSL (Scala)	generators, compiled through FIRRTL and CIRCT	UC Berkeley, 2012	Verilog, SystemVerilog
SpinalHDL	eDSL (Scala)	generators	Charles Papon, 2015	VHDL, Verilog
DFiant (DFHDL)	eDSL (Scala 3)	dataflow, timing-agnostic description	Technion, 2017	Verilog, VHDL
Amaranth	eDSL (Python)	generators, successor of Migen (first called nMigen)	2019, renamed in 2021	Verilog via Yosys
Migen	eDSL (Python)	generators, still the base of the LiteX SoC builder	M-Labs, 2011	Verilog
PyMTL3	eDSL (Python)	modeling at functional, cycle, and RTL level	Cornell University, 2014 (as PyMTL)	Verilog
PyRTL	eDSL (Python)	minimal RTL for teaching and research	UC Santa Barbara, 2014	Verilog
Magma	eDSL (Python)	strongly typed circuits	Stanford University, 2017 (inactive since 2024)	Verilog via CoreIR
Kratos	eDSL (Python)	generators with source-level debugging	Stanford University, 2019 (inactive since 2023)	SystemVerilog
HWT	eDSL (Python)	hardware construction library	Michal Orsák, 2015	VHDL, Verilog, SystemC
[illegible]	[illegible]	[illegible] with [illegible] linter, and package manager	[illegible] since 2020 (open source)	[illegible]

All non-standard languages have one thing in common: their output is Verilog or SystemVerilog (sometimes VHDL), which then goes through the same simulators and synthesis tools as hand-written RTL. (System)Verilog is the common exchange format of the whole industry. Table 7 compares the four families.

Table 7: Comparison of the HDL families

Criterion	Standardized HDLs	Embedded DSLs	New standalone HDLs	HLS
Industry adoption	very high, the reference for all EDA tools, IP, and jobs	niche, some companies and many open-source cores (Rocket Chip and BOOM in Chisel, VexRiscv in SpinalHDL)	research and early adopters	established for data paths in commercial FPGA and ASIC flows
Tool support	all simulators, synthesis, lint, formal, and timing tools	host-language toolchain plus the generated Verilog	one compiler, few other tools	mostly commercial, open-source tools less mature
Abstraction and productivity	RTL, limited metaprogramming (parameters, generate)	high, the full host language builds generators	medium to high, modern syntax and type inference	highest, algorithmic description
Learning curve	moderate, Verilog has many pitfalls	host language plus hardware thinking	new syntax, small community and documentation	low for software developers, but good results need hardware knowledge and tool directives
Error checking	VHDL strongly typed, (System)Verilog weakly typed and dependent on lint	type systems of the host languages catch many errors	strong type systems, some prevent whole classes of bugs (e.g., races in Bluespec)	depends on the C model, timing is implicit
Debugging	source, waveform, and netlist correspond directly	generated Verilog with machine-made names	similar to eDSLs	the RTL is hard to relate to the source, latency depends on scheduling
Long-term risk	decades of stability	depends on small projects, several are dormant	early stage	lock-in to a vendor tool

2.3.1 Why SystemVerilog in This Course

- **Industry standard.** SystemVerilog (IEEE 1800-2023 [6]) is the dominant language for ASIC design and verification. Every EDA tool, IP vendor, and foundry flow supports it.
- **Open-source tool support.** All tools of our flow read it: the simulators Verilator and Icarus Verilog (also under cocotb), Yosys and LibreLane, slang, and Verible. The AMS chip template (Section 1.10) is written in SystemVerilog.
- **You have to read it anyway.** Netlists, gate-level simulations, and the output of every alternative HDL are (System)Verilog. Debugging always ends there. Section 2.5.13 shows how plain Verilog maps onto SystemVerilog.
- **cocotb for verification.** The verification part of SystemVerilog (classes and constrained randomization) and the UVM library built on it form a large field of their own. With cocotb (Section 4), we write testbenches in Python and use SystemVerilog mainly for the synthesizable RTL.
- **A good foundation.** With solid RTL skills in SystemVerilog, you can learn an embedded DSL such as Chisel or Amaranth later with little extra effort, since it describes the same registers and logic.

2.3.2 VHDL versus SystemVerilog

Table 8 compares the two languages.

Table 8: VHDL versus SystemVerilog

Aspect	VHDL	SystemVerilog
Standard	IEEE 1076-2019	IEEE 1800-2023, includes Verilog
Typing	strong, explicit conversions (`numeric_std`)	weak, widths are extended or truncated silently, lint catches part of it
Verbosity	verbose (entity and architecture, type conversions)	compact
Case sensitivity	case-insensitive	case-sensitive (`clk` and `Clk` are different)
Simulation semantics	signals update after a delta cycle, the order of processes never matters	blocking and non-blocking assignments, races are possible if the coding rules are broken
Verification features	libraries (OSVVM, UVVM)	built in: classes, randomization, assertions (SVA), coverage, plus the UVM class library (IEEE 1800.2)
Typical use	FPGA, aerospace and defense, Europe, academia	ASIC design and verification worldwide
Open-source tools	GHDL (simulation), experimental synthesis via a Yosys plugin	Verilator, Icarus Verilog, Yosys, slang, Verible

In short, VHDL protects you with strong typing, while SystemVerilog relies on coding discipline and on the linter. The rest of this chapter is about that discipline.

2.4 Cheat-Sheets, Guidelines, and Further Reading

Besides the SystemVerilog cheat-sheet in the appendix of this book (Section 12), two cheat-sheets are provided with the AMS chip template:

- SystemVerilog Cheat-Sheet (S. H. Shawon, Algo Science Lab)
- Verilog Cheat-Sheet (S. Winberg and J. Taylor)

> ⚠ Using the Cheat-Sheets
>
> Both cheat-sheets are syntax references, not style guides. The SystemVerilog cheat-sheet also covers the verification part of the language (classes, randomization, interfaces, assertions), and not all of its examples are synthesizable or accepted by the open-source tools of our flow. The Verilog cheat-sheet describes plain Verilog-2001 with `reg`, `wire`, and `always @(...)`, for which we use the SystemVerilog constructs of this chapter instead (Section 2.5.13). Whatever you copy, lint it with Verilator and check it with a quick Yosys synthesis (Section 2.7).

For HDL coding guidelines, read the Circuit Designer's Etiquette of the Analog Circuit Design course, in particular its section VHDL/Verilog Coding Guide. Section 2.9 summarizes its rules for SystemVerilog. The etiquette recommends the detailed lowRISC Verilog Coding Style Guide [7], from which this course takes most of its naming conventions.

> ℹ Further Reading
>
> - The SystemVerilog standard IEEE 1800-2023 [6] is the final reference. It is available free of charge through the IEEE GET Program (announcement by Accellera).
> - [1] shows SystemVerilog and VHDL side by side for all basic building blocks, which helps with the transition.
> - [8] covers RTL modeling with SystemVerilog for simulation and synthesis in depth.
> - [9] collects 101 common coding errors in Verilog and SystemVerilog and how to avoid them.

2.5 SystemVerilog for VHDL Designers

Table 9 maps the most important VHDL constructs onto SystemVerilog. The following sections go through them one by one. Where a VHDL version helps, it comes first, and the sequential designs use the two-process style you know from your VHDL courses.

Table 9: Mapping of VHDL constructs onto SystemVerilog

Concept	VHDL	SystemVerilog
Design unit	`entity` + `architecture`	`module ... endmodule`
Generic / parameter	`WIDTH : positive := 8`	`parameter int unsigned WIDTH = 8`
Constant	`constant C_LAST : natural := 9;`	`localparam int unsigned LAST = 9;`
Port	`a_i : in std_logic_vector(7 downto 0)`	`input logic [7:0] a_i`
Bit, vector	`std_logic`, `std_logic_vector`	`logic`, `logic [N-1:0]`
Unsigned, signed	`unsigned`, `signed` (`numeric_std`)	`logic [N-1:0]` (unsigned), `logic signed [N-1:0]`
All zeros	`(others => '0')`	`'0`
Concatenation	`a & b`	`{a, b}`
Concurrent assignment	`y <= a and b;`	`assign y = a & b;`
Conditional assignment	`y <= a when s = '1' else b;`	`assign y = s ? a : b;`
Combinational process	`process(a, b)` (VHDL-2008: `process(all)`)	`always_comb`
Register process	`process(rst_i, clk_i)` with `rising_edge(clk_i)`	`always_ff @(posedge clk_i or negedge rst_ni)`
Signal assignment in the register process	`q <= d;`	`q <= d;` (non-blocking)
Variable assignment	`v := a;`	`v = a;` (blocking)
Enumeration	`type t_state is (IDLE, RUN, DONE);`	`typedef enum logic [1:0] {StIdle, StRun, StDone} state_e;`
Record	`type t_x is record ... end record;`	`typedef struct packed { ... } x_t;`
Package	`package ... end package;`, `use work.pkg.all;`	`package ... endpackage`, `pkg::NAME`
Instance	`adder_0: entity work.adder port map(...)`	`adder adder_0 (...);`
Generate	`gen: for i in 0 to 3 generate`	`for (genvar i = 0; i < 4; i++) begin : gen`
Width of a constant	`natural(ceil(log2(real(N))))`	`$clog2(N)`
Comment	`--`	`//` and `/* ... */`

2.5.1 Modules, Ports, and Parameters

An unsigned adder with a generic width and a carry bit is a good first example. In VHDL, it needs an entity, an architecture, three type conversions, and a `resize`:

```
-- SPDX-FileCopyrightText: 2026 Simon Dorrer
-- SPDX-License-Identifier: Apache-2.0
-- Description: Unsigned adder with a generic width and a carry bit.

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity adder is
  generic(
    WIDTH : positive := 8
  );
  port(
    a_i   : in  std_logic_vector(WIDTH - 1 downto 0);
    b_i   : in  std_logic_vector(WIDTH - 1 downto 0);
    sum_o : out std_logic_vector(WIDTH downto 0)
  );
end entity adder;

architecture rtl of adder is
begin
  sum_o <= std_logic_vector(resize(unsigned(a_i), WIDTH + 1)
                            + unsigned(b_i));
end architecture rtl;
```

In SystemVerilog, a single `module` replaces entity and architecture:

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Unsigned adder with a parameterized width and a carry bit.

`default_nettype none

module adder #(
  parameter int unsigned WIDTH = 8
) (
  input  logic [WIDTH-1:0] a_i,
  input  logic [WIDTH-1:0] b_i,
  output logic [WIDTH:0]   sum_o
);

  assign sum_o = {1'b0, a_i} + {1'b0, b_i};
```

```
endmodule

`default_nettype wire
```

- **Parameters** are declared in the `#( ... )` list before the ports, like generics. The type `int unsigned` makes the parameter a non-negative 32-bit integer. Constants that must not be changed from outside are `localparam`s.
- **Ports** are declared in the module header (ANSI style) with direction, type, width, and name. We use the suffixes `_i`, `_o`, and `_io` for inputs, outputs, and bidirectional ports.
- **No type conversions.** SystemVerilog has no separate types for bit vectors and numbers, so `+` works directly on `logic` vectors. The concatenations `{1'b0, a_i}` and `{1'b0, b_i}` make the extension to `WIDTH+1` bits explicit. The carry would be kept without them as well, since the target is `WIDTH+1` bits wide (Section 2.5.3), but the explicit version states the intent.
- **Case sensitivity.** Unlike VHDL, SystemVerilog is case-sensitive: `Clk` and `clk` are two different signals. Write signals in lowercase snake_case and parameters in UPPERCASE.
- **`` `default_nettype none ``** at the top of the file makes an undeclared name an error. Without it, Verilog silently creates a 1-bit wire for a misspelled name on the left side of an `assign` or in a port connection. The `` `default_nettype wire `` at the end restores the default for files compiled afterwards.
- **One module per file**, and the file name equals the module name (`adder.sv`). Pass all files to the tools as a list instead of pulling modules in with `` `include ``, so that the list shows every file of the design and no module is read twice. Even in a list, a `` `define `` stays active for all files that follow, in every tool. Icarus Verilog also keeps `` `default_nettype ``, while Verilator and Yosys reset it for each file. Ending every file with `` `default_nettype wire `` gives the same result in all tools.

2.5.2 Data Types and Literals

The everyday type of SystemVerilog is `logic`. It has four values: `0`, `1`, `x` (unknown), and `z` (high impedance). VHDL's `std_logic` has nine values, but the four that matter in RTL map directly. As in VHDL, where an uninitialized `std_logic` is `'U'`, a `logic` that is never assigned or reset is `x` in a four-state simulator such as Icarus Verilog, so a missing reset stays visible in the waveform. Verilator is a two-state simulator for speed and replaces `x` with `0` or a random value, which is one reason to simulate with both.

- **Use `logic` for every signal in RTL.** A `logic` variable assigned in an `always_comb` block, an `always_ff` block, or by an `assign` must not be assigned anywhere else. Verilator reports a second driver as a `MULTIDRIVEN` warning. Icarus Verilog reports an error only if one of the drivers is an `assign`, and it accepts two `always_comb` or two `always_ff` blocks without a message. The single-driver rule corresponds to the unresolved `std_ulogic` in VHDL, not to the resolved `std_logic`, which allows several drivers. Nets with several drivers (tri-state buses) need the type `wire` and only appear at the pads of an ASIC.
- **`wire` and `reg` are legacy Verilog.** Despite its name, `reg` does not mean register. Whether a flip-flop is created depends only on how a variable is assigned, never on its type.

- **Vectors** are declared as `logic [7:0] data`, like `std_logic_vector(7 downto 0)` with the MSB on the left. Select a bit with `data[0]`, a fixed range with `data[7:4]`, and a range at a variable position with `data[i*4 +: 4]` (four bits starting at bit `i*4`).
- **Signedness.** Vectors are unsigned by default, `logic signed [15:0] acc` declares a signed one, and `$signed()` and `$unsigned()` convert. If only one operand of an expression is unsigned, the whole expression is evaluated unsigned, so do not mix them.
- **Two-state types** such as `bit`, `int` (32 bits, signed), and `int unsigned` know only `0` and `1`. Use them for parameters, loop variables, and in testbenches, but not for RTL signals, where they would turn an `x` into a `0` and hide bugs.
- **Arrays.** A *packed* array `logic [3:0][7:0] word` is a 32-bit vector made of four bytes and can be used as one vector. An *unpacked* array `logic [7:0] mem [16]` holds 16 separate bytes, e.g., a register file.

Literals carry their width in front of the base, as Table 10 shows. Always size your literals, because an unsized number such as `20` is a 32-bit integer.

Table 10: Literals in SystemVerilog and VHDL

SystemVerilog	VHDL	Meaning
`8'b1010_0101`	`"10100101"`	8-bit binary, `_` only improves readability
`8'hA5`	`x"A5"`	8-bit hexadecimal
`8'd165`	`std_logic_vector(to_unsigned(165, 8))`	8-bit decimal
`'0`, `'1`	`(others => '0')`, `(others => '1')`	all bits zero or one, for any width
`'x`, `'z`	`(others => 'X')`, `(others => 'Z')`	all bits unknown or high impedance
`4'b10x1`	`"10X1"`	literal with an unknown bit
`165`	`165`	unsized: a 32-bit signed integer

2.5.3 Operators and Expression Widths

Most operators exist in both languages, but SystemVerilog uses the C notation (Table 11).

Table 11: Operators in VHDL and SystemVerilog

VHDL	SystemVerilog	Meaning
`not a`	`~a`	bitwise NOT
`a and b`, `a or b`, `a xor b`	`a & b`, `a | b`, `a ^ b`	bitwise AND, OR, XOR
`and a` (VHDL-2008 only)	`&a`, `|a`, `^a`	reduction: AND, OR, XOR (parity) of all bits, 1-bit result
`not`, `and`, `or` on `boolean`	`!a`, `a && b`, `a || b`	logical operators, 1-bit result
`=`, `/=`	`==`, `!=`	equality, `x` if an `x` or `z` bit makes the result ambiguous
none	`===`, `!==`	case equality, compares `x` and `z` literally, only in testbenches
`<`, `<=`, `>`, `>=`	`<`, `<=`, `>`, `>=`	relational (`<=` is also the non-blocking assignment)
`+`, `-`, `*`	`+`, `-`, `*`	arithmetic
`shift_left()`, `shift_right()`	`<<`, `>>`	logical shift
`shift_right()` on `signed`	`>>>`	arithmetic shift, only for signed operands
`a & b`	`{a, b}`	concatenation
none	`{4{a}}`	replication: `a` four times
`a when s = '1' else b`	`s ? a : b`	conditional operator

The biggest difference from VHDL is how widths are handled. In SystemVerilog, the operands of an expression are extended to the width of the widest operand *and of the assignment target*, and the result is then truncated to the target. Nothing is reported as an error:

```
logic [7:0] a, b, sum8;
logic [8:0] sum9;
logic [3:0] nibble, count;
logic       hit;

assign sum8   = a + b;          // carry silently lost, no warning
assign sum9   = a + b;          // carry kept, the target is 9 bits wide
assign nibble = a;              // MSBs cut off (Verilator: WIDTHTRUNC)
assign hit    = (count == 20);  // always false (Verilator: WIDTHEXPAND)
```

In VHDL, `sum9 <= a + b` and `nibble <= a` are length mismatches: GHDL already warns about `nibble <= a` during the analysis and stops the simulation with a bound check failure. `sum8` loses the carry in both languages. Verilator reports the truncation and the comparison with a

constant that does not fit, but not the lost carry. So think about the width of every arithmetic result, and make extensions explicit with a concatenation such as `{1'b0, a}` or a size cast such as `9'(a)`.

2.5.4 Concurrent Assignments

The VHDL concurrent signal assignment becomes an `assign` statement, a *continuous assignment* that is re-evaluated whenever one of its operands changes:

```
// VHDL: y_o <= a_i and b_i;
assign y_o   = a_i & b_i;

// VHDL: mux_o <= a_i when sel_i = '1' else b_i;
assign mux_o = sel_i ? a_i : b_i;
```

A selected signal assignment (`with ... select`) becomes a `case` statement in an `always_comb` block (Section 2.5.8). Unlike VHDL-93, SystemVerilog lets you read an output port inside the module, so the internal copy of an output that VHDL needs is not necessary.

> Exercise: A 4:1 Multiplexer in SystemVerilog
>
> The following VHDL entity is a 4:1 multiplexer for 8-bit data with a zero flag:
>
> ```
> -- SPDX-FileCopyrightText: 2026 Simon Dorrer
> -- SPDX-License-Identifier: Apache-2.0
> -- Description: 4:1 multiplexer for 8-bit data with a zero flag.
>
> library ieee;
> use ieee.std_logic_1164.all;
>
> entity mux4 is
>   port(
>     a_i    : in  std_logic_vector(7 downto 0);
>     b_i    : in  std_logic_vector(7 downto 0);
>     c_i    : in  std_logic_vector(7 downto 0);
>     d_i    : in  std_logic_vector(7 downto 0);
>     sel_i  : in  std_logic_vector(1 downto 0);
>     y_o    : out std_logic_vector(7 downto 0);
>     zero_o : out std_logic
>   );
> end entity mux4;
>
> architecture rtl of mux4 is
>   signal y : std_logic_vector(7 downto 0);
> begin
>   with sel_i select
>     y <= a_i when "00",
> ```

```
          b_i when "01",
          c_i when "10",
          d_i when others;

  y_o <= y;
  zero_o <= '1' when y = x"00" else '0';
end architecture rtl;
```

1. Translate it into a SystemVerilog module `mux4` with the same port names and save it as `mux4.sv`. Section 2.5.8 shows the syntax of the `case` statement.
2. Lint your module in the container with `verilator --lint-only -Wall mux4.sv` (Section 2.7.2 explains the command). Verilator must not report any warning.

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

Solution: A 4:1 Multiplexer in SystemVerilog

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: 4:1 multiplexer for 8-bit data with a zero flag.

`default_nettype none

module mux4 (
  input  logic [7:0] a_i,
  input  logic [7:0] b_i,
  input  logic [7:0] c_i,
  input  logic [7:0] d_i,
  input  logic [1:0] sel_i,
  output logic [7:0] y_o,
  output logic       zero_o
);

  always_comb begin
    case (sel_i)
      2'b00:   y_o = a_i;
      2'b01:   y_o = b_i;
      2'b10:   y_o = c_i;
      default: y_o = d_i;
    endcase
  end
```

```
  // An output can be read inside the module, no internal copy needed
  assign zero_o = (y_o == 8'h00);

endmodule

`default_nettype wire
```

- The selected signal assignment becomes a `case` statement in `always_comb`, and `when others` becomes `default`. A chain of conditional operators `?:` in an `assign` is also correct.
- `x"00"` becomes `8'h00`. The comparison already has a 1-bit result, so `'1' when ... else '0'` is not needed.
- The output `y_o` is read directly, and the internal signal `y` of the VHDL version is gone.
- The zero flag can also be written with the reduction NOR operator, `assign zero_o = ~|y_o;`.

2.5.5 Processes: `always_comb`, `always_ff`, and `always_latch`

SystemVerilog has three dedicated process types for synthesizable code, which replace the general-purpose `always` block of Verilog:

- **`always_comb`** describes combinational logic. It is sensitive to every signal it reads, like `process(all)` in VHDL-2008, so the sensitivity list can never be incomplete. The tools also check that the block is really combinational: if a variable is not assigned on every path, Verilator warns (`LATCH`) and Yosys stops with an error.
- **`always_ff @(posedge clk_i or negedge rst_ni)`** describes flip-flops. Its event list contains only the clock edge and, for an asynchronous reset, the reset edge.
- **`always_latch`** describes an intended latch, which you will rarely need.

Within these blocks, SystemVerilog has two kinds of assignments, which correspond to the variables and signals of VHDL:

- The **blocking assignment** `=` updates the variable immediately, and the next statement already sees the new value, like a VHDL variable (`:=`).
- The **non-blocking assignment** `<=` evaluates the right-hand side immediately but updates the variable only after all blocks triggered by the same clock edge have read their inputs, like a VHDL signal assignment.

The difference matters as soon as a register reads another register that is updated at the same clock edge. The following two registers swap their values at every clock edge:

```
always_ff @(posedge clk_i) begin
  a_q <= b_q;
  b_q <= a_q;  // reads the old value of a_q: a real swap
end
```

With blocking assignments in this block, `b_q` would read the *new* value of `a_q`, and both registers would end up with the old value of `b_q`, in simulation and in synthesis. It gets worse when the two blocking assignments sit in two separate `always_ff` blocks. The simulation result then depends on the order in which the simulator runs the blocks, which the standard leaves open, while synthesis builds the swap anyway. Icarus Verilog 14 and Verilator 5.052 both show no swap for this case, while the netlist of Yosys swaps. This mismatch between simulation and synthesis is one of the most common SystemVerilog bugs, and the following rules avoid it.

! The Golden Rules of Processes

1. In `always_ff`, use only non-blocking assignments `<=`.
2. In `always_comb`, use only blocking assignments `=`.
3. Assign every variable in exactly one block (or one `assign`).
4. Never use the plain `always` block or `initial` in RTL, only `always_comb`, `always_ff`, and `always_latch`.

2.5.6 Registers and Resets

In your VHDL style, every register process has an active-high asynchronous reset (Section 2.2.2). In SystemVerilog, the three register variants you need look as follows:

```
// Asynchronous active-low reset (the default in this course)
always_ff @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) begin
    q <= '0;
  end else begin
    q <= d;
  end
end

// Synchronous active-high reset (counter of the AMS chip template)
always_ff @(posedge clk_i) begin
  if (reset_i) begin
    q <= '0;
  end else begin
    q <= d;
  end
end

// No reset (pure data path registers)
always_ff @(posedge clk_i) begin
  q <= d;
end
```

In this course, the default is an **asynchronous active-low reset** named `rst_ni` (`_n` for active low, `_i` for input). This is the common convention in ASIC design and the rule of the lowRISC style guide. It also matches the hardware: the reset flip-flops of the SG13CMOS5L standard-

cell library, e.g., `sg13cmos5l_dfrbp_1` and `sg13cmos5l_dfrbpq_1`, have an active-low reset pin `RESET_B`, and the reset pad of a chip is usually active low (`rst_n` in the chip core of the AMS chip template). Synthesis handles either polarity: an active-high reset only costs an inverter.

- The reset test must be the first statement in the block, and its polarity must match the event list (`negedge rst_ni` with `if (!rst_ni)`). Otherwise, the block cannot be mapped onto a flip-flop with an asynchronous reset.
- An asynchronous reset may be asserted at any time, but it must be released synchronously to the clock with a reset synchronizer (Section 9).
- A synchronous reset is ordinary logic in front of the D input of a flip-flop without reset.
- A clock enable, written as `end else if (en_i) begin q <= d; end`, keeps the old value when `en_i` is low. The synthesis tool maps it onto a flip-flop with an enable input or a multiplexer in front of D. This is a register, not a latch.

2.5.7 The Two-Process Style

In your VHDL designs, you split each block into a register process (`reg_proc`), which only holds the reset values and copies the `next_` signals into the registers, and one or more combinational processes, which start with complete default assignments and contain all decisions (Section 2.2.2). This style maps one to one onto SystemVerilog, and we keep it: `always_ff` is the register process, `always_comb` the combinational process, and the VHDL pair `next_state` and `state` becomes `state_d` (the next value at the D input of the register in Figure 4) and `state_q` (the current value at its Q output), as in the lowRISC style guide. Here is the toggle flip-flop of Section 2.2.2 in SystemVerilog:

```
// Register process: only clock and reset in the event list
always_ff @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) begin
    state_q <= 1'b0;
  end else begin
    state_q <= state_d;
  end
end

// Combinational process: the sensitivity list is inferred
always_comb begin
  // default assignment
  state_d = state_q;

  // calculate state_d from state_q and the inputs
  if (toggle_i) begin
    state_d = ~state_q;
  end
end
```

The three rules of the VHDL style carry over. `always_comb` makes the second one unnecessary, and the tools check the other two:

- The event list of `always_ff` contains only the clock edge and the reset edge, just like the sensitivity list of `reg_proc`. If you add a data signal to it, Yosys stops with an error, even though Verilator accepts it.
- `always_comb` infers the complete sensitivity list by itself, so a forgotten signal is no longer possible.
- The default assignment `state_d = state_q;` comes first, as in VHDL. If you forget it, the tools report the latch: Verilator warns, and Yosys stops with an error (Section 2.5.8).

The register process uses non-blocking assignments `<=` and the combinational process blocking assignments `=`, as the Golden Rules of Section 2.5.5 demand. The order of the two blocks in the file does not matter, since both are concurrent. As in your VHDL designs, all listings of this chapter put the register process first and the combinational process second. Yosys 0.70 synthesizes the toggle flip-flop into exactly two cells, an inverter and a flip-flop with enable (`$_DFFE_PN0P_`) whose enable input is `toggle_i`. The default assignment that keeps the state has become the enable of the flip-flop, not a latch.

As a larger example, consider a tick generator: while `en_i` is high, `tick_o` is high for one clock cycle every `DIV` clock cycles. Such a one-cycle pulse is a *strobe*, which typically triggers other blocks, e.g., to restart a counter or to signal that a result is valid. Here is the VHDL version in the style you know:

```
-- SPDX-FileCopyrightText: 2026 Simon Dorrer
-- SPDX-License-Identifier: Apache-2.0
-- Description: Tick generator in the two-process style. While
-- en_i is high, tick_o is high for one clock cycle every DIV clock
-- cycles (DIV >= 2).

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.math_real.all;

entity tick_gen is
  generic(
    DIV : positive := 10
  );
  port(
    clk_i  : in  std_logic;
    rst_i  : in  std_logic;
    en_i   : in  std_logic;
    tick_o : out std_logic
  );
end entity tick_gen;

architecture rtl of tick_gen is
  constant C_COUNT_WIDTH : natural := natural(ceil(log2(real(DIV))));

  subtype t_count is unsigned(C_COUNT_WIDTH - 1 downto 0);
```

```
  constant C_LAST : t_count := to_unsigned(DIV - 1, C_COUNT_WIDTH);

  signal count      : t_count;
  signal next_count : t_count;
  signal tick       : std_logic;
  signal next_tick  : std_logic;
begin
  -- Register process: reset values and copies of the next_ signals only
  reg_proc: process(rst_i, clk_i) is
  begin
    if rst_i = '1' then
      count <= (others => '0');
      tick <= '0';
    elsif rising_edge(clk_i) then
      count <= next_count;
      tick <= next_tick;
    end if;
  end process reg_proc;

  -- Combinational process: defaults first, then all decisions
  counter_logic: process(count, en_i) is
  begin
    next_count <= count;
    next_tick <= '0';
    if en_i = '1' then
      if count = C_LAST then
        next_count <= (others => '0');
        next_tick <= '1';
      else
        next_count <= count + 1;
      end if;
    end if;
  end process counter_logic;

  tick_o <= tick;
end architecture rtl;
```

And the same design in SystemVerilog:

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Tick generator in the two-process style. While
// en_i is high, tick_o is high for one clock cycle every DIV clock
// cycles (DIV >= 2).
```

```
`default_nettype none

module tick_gen #(
  parameter int unsigned DIV = 10
) (
  input  logic clk_i,
  input  logic rst_ni,
  input  logic en_i,
  output logic tick_o
);

  localparam int unsigned      COUNT_WIDTH = $clog2(DIV);
  localparam [COUNT_WIDTH-1:0] COUNT_LAST  = COUNT_WIDTH'(DIV - 1);

  logic [COUNT_WIDTH-1:0] count_d, count_q;
  logic                   tick_d, tick_q;

  // Register process: reset values and copies of the _d signals only
  always_ff @(posedge clk_i or negedge rst_ni) begin
    if (!rst_ni) begin
      count_q <= '0;
      tick_q  <= 1'b0;
    end else begin
      count_q <= count_d;
      tick_q  <= tick_d;
    end
  end

  // Combinational process: defaults first, then all decisions
  always_comb begin
    count_d = count_q;
    tick_d  = 1'b0;
    if (en_i) begin
      if (count_q == COUNT_LAST) begin
        count_d = '0;
        tick_d  = 1'b1;
      end else begin
        count_d = count_q + 1'b1;
      end
    end
  end

  assign tick_o = tick_q;

endmodule
```

```
`default_nettype wire
```

Table 12 lists the correspondences. With the default `DIV = 10`, both versions describe the same hardware apart from the reset polarity: four flip-flops for the counter, one for the tick, and the increment and compare logic in front of them. The VHDL and SystemVerilog versions of the tick generator and of the two FSMs of this chapter were simulated with the same cocotb testbenches, with the reset adapted, to make sure they behave identically.

Table 12: The two-process style in VHDL and SystemVerilog

VHDL (two-process)	SystemVerilog (two-process)
`reg_proc: process(rst_i, clk_i)`	`always_ff @(posedge clk_i or negedge rst_ni)`
`counter_logic: process(count, en_i)`	`always_comb` (complete sensitivity, inferred)
`count`, `next_count`	`count_q`, `count_d`
`if rst_i = '1' then` (active high)	`if (!rst_ni)` (active low)
`natural(ceil(log2(real(DIV))))`	`$clog2(DIV)`
`to_unsigned(DIV - 1, C_COUNT_WIDTH)`	`COUNT_WIDTH'(DIV - 1)` (size cast)
`(others => '0')`	`'0`
`count + 1`	`count_q + 1'b1`
`tick_o <= tick;`	`assign tick_o = tick_q;`

The two-process style has three advantages. The register process is a fixed template that you never have to think about again. The defaults at the top of the combinational process make latches impossible. And the next values (`_d`) are signals that you can inspect in the waveform and reuse in other logic. The single-process style, in which the next-state logic is written directly into `always_ff` (as in the counter of the AMS chip template, Section 2.8), is fine for simple registers and counters, and both styles synthesize to the same hardware. We recommend the two-process style whenever the next-state logic is more than a line or two, and always for FSMs.

2.5.8 `if`, `case`, and Latches

`if`, `else if`, and `else` work as `if`, `elsif`, and `else` in VHDL and describe a priority. Unlike in VHDL, a condition does not need to be of type `boolean`: every nonzero value is true, so you write `if (en_i)` instead of `if en_i = '1' then`. The `case` statement works as in VHDL, with `default` instead of `when others`. There are two important differences:

- VHDL requires a `case` to cover all values, but SystemVerilog does not. A missing value in an `always_comb` block leaves the variable unassigned for that value, and the tools infer a **latch** that holds the old value.
- The qualifier **`unique`** in `unique case` states that exactly one item matches at a time, so the items can be implemented as a parallel multiplexer instead of a priority chain. Simulators that support it report a violation at run time. Icarus Verilog prints a “sorry” message that it

ignores the qualifier, but it still warns at run time when no item matches. It does not check for overlapping items. Use `unique case` together with a `default` item.

For decoders, `casez` treats `?` in the case items as don't care, e.g., `4'b01??`. Avoid `casex`, which also treats an `x` in the case expression as don't care and thereby hides `x` values in simulation.

A latch appears whenever a combinational block does not assign a variable on every path:

```
// Latch: y_o keeps its old value while en_i is low
always_comb begin
  if (en_i) y_o = a_i;
end

// Fixed: default assignment first, then the decisions
always_comb begin
  y_o = '0;
  if (en_i) y_o = a_i;
end
```

In a synchronous design, latches are almost always a bug: static timing analysis treats them differently from flip-flops (Section 5), they pass glitches, and they complicate the scan test (Section 8). The SG13CMOS5L library contains latch cells (e.g., `sg13cmos5l_dlhq_1`), so an unintended latch synthesizes without complaint if nobody checks. Here `always_comb` pays off: for the plain Verilog block `always @(*)`, Yosys 0.70 only prints a warning and builds the latches, while it stops with an error for `always_comb`. If you really need a latch, write it with `always_latch`, so that your intention is documented.

Exercise: Let the Linter Find the Bugs

The following module is meant to be a small ALU with a registered result: `op_i` selects addition (`00`), subtraction (`01`), bitwise AND (`10`), or bitwise OR (`11`). `result_o` is the registered result, `zero_o` is high when `result_o` is zero, and the register has an asynchronous active-low reset. The file contains several mistakes.

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Small ALU with a registered result. This file contains
// mistakes on purpose (exercise), do not use it as a template.

module lint_me (
  input  logic       clk_i,
  input  logic       rst_ni,
  input  logic [1:0] op_i,
  input  logic [7:0] a_i,
  input  logic [7:0] b_i,
  output logic [7:0] result_o,
  output logic       zero_o
```

```
);

  logic [7:0] alu;
  logic [7:0] result_q;

  // Result register
  always_ff @(posedge clk_i or negedge rst_ni) begin
    if (!rst_ni) begin
      result_q = 8'h00;
    end else begin
      result_q = alu;
    end
  end

  // ALU: 00 add, 01 subtract, 10 AND, 11 OR
  always_comb begin
    case (op_i)
      2'b00: alu = a_i + b_i;
      2'b01: alu = a_i - b_i;
      2'b10: alu = a_i & b_i;
    endcase
  end

  assign result_o = result_q;
  assign zero = (result_q == 8'h00);

endmodule
```

1. Save the file as `lint_me.sv` and lint it with `verilator --lint-only -Wall lint_me.sv`. Explain every warning.
2. Synthesize the file with `yosys -p "read_verilog -sv lint_me.sv; synth -top lint_me; stat"`. What does Yosys report?
3. Fix the code until Verilator reports no warning, and synthesize it again. Which cells hold the result?

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

Solution: Let the Linter Find the Bugs

Verilator 5.052 reports seven warnings and exits with an error:

- `IMPLICIT`: the name `zero` is not declared, so Verilog creates an implicit 1-bit wire. It is a typo for `zero_o`.
- `UNDRIVEN` for `zero_o` and `UNUSEDSIGNAL` for `zero`: two consequences of the same typo.

- `BLKSEQ` (twice): blocking assignments `=` in the `always_ff` block, which must be non-blocking `<=`.
- `CASEINCOMPLETE`: the case does not cover `op_i = 2'b11`, the OR operation was forgotten.
- `LATCH` for `alu`: because of the missing case item, `alu` keeps its old value for `op_i = 2'b11`.

The corrected module fixes the typo, adds `` `default_nettype none ``, which would have turned it into an error, completes the case with `unique` and a `default`, and uses non-blocking assignments:

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Small ALU with a registered result (corrected version).

`default_nettype none

module lint_me (
  input  logic       clk_i,
  input  logic       rst_ni,
  input  logic [1:0] op_i,
  input  logic [7:0] a_i,
  input  logic [7:0] b_i,
  output logic [7:0] result_o,
  output logic       zero_o
);

  logic [7:0] alu;
  logic [7:0] result_q;

  // Result register
  always_ff @(posedge clk_i or negedge rst_ni) begin
    if (!rst_ni) begin
      result_q <= 8'h00;
    end else begin
      result_q <= alu;
    end
  end

  // ALU: 00 add, 01 subtract, 10 AND, 11 OR
  always_comb begin
    unique case (op_i)
      2'b00:   alu = a_i + b_i;
      2'b01:   alu = a_i - b_i;
      2'b10:   alu = a_i & b_i;
      default: alu = a_i | b_i;  // 2'b11
    endcase
  end
```

```
  assign result_o = result_q;
  assign zero_o   = (result_q == 8'h00);

endmodule

`default_nettype wire
```

For the original file, Yosys warns that `zero` is implicitly declared and then stops with `ERROR: Latch inferred for signal `\lint_me.\alu' from always_comb process`. The fixed version synthesizes to 114 generic cells, among them eight `$_DFF_PN0_` cells for `result_q`: flip-flops on the positive clock edge (`P`) with an active-low reset (`N`) to zero (`0`).

2.5.9 Finite State Machines

FSMs are where the two-process style pays off most. As an example, consider a simple controller: a `start_i` pulse moves it from idle to a run state, in which `busy_o` is high, until `last_i` is high. Then `done_o` is high for one clock cycle, and the FSM returns to idle. Both outputs depend only on the state (Moore FSM). Figure 7 shows the state diagram, with the outputs written inside the states.

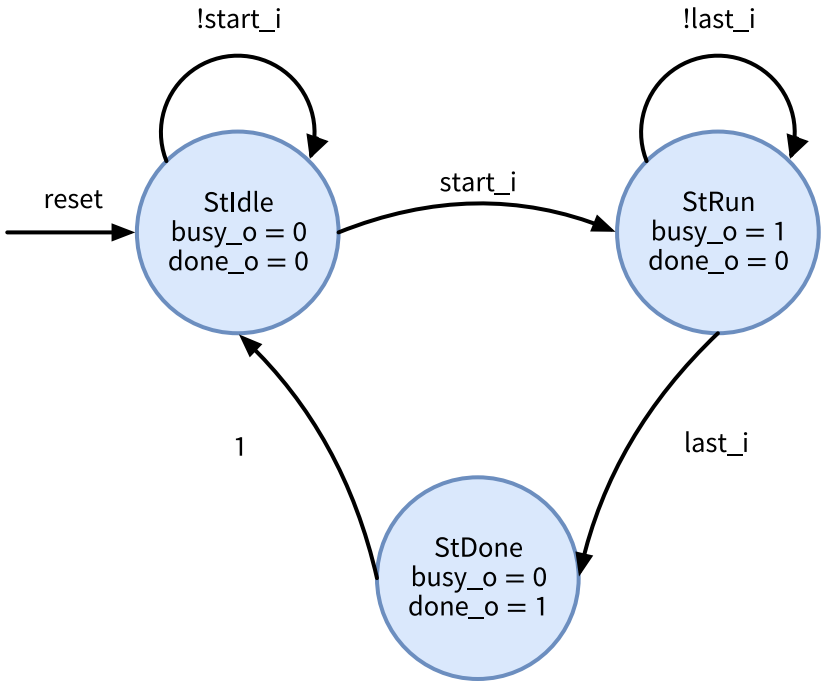


Figure 7: State diagram of the controller (Moore FSM). The outputs belong to the states and are written inside them. An arc shows the condition under which the FSM takes it at the next rising clock edge, and `1` means always. The VHDL version names the states `CTRL_IDLE`, `CTRL_RUN`, and `CTRL_DONE`. The unused encoding `2'b11` of the SystemVerilog version, which also leads back to `StIdle`, is not drawn.

In VHDL, the controller looks as follows:

```vhdl
-- SPDX-FileCopyrightText: 2026 Simon Dorrer
-- SPDX-License-Identifier: Apache-2.0
-- Description: Moore FSM of a simple controller. A start_i pulse
-- moves it from idle to run (busy_o high) until last_i, then done_o
-- is high for one clock cycle before the FSM returns to idle.

library ieee;
use ieee.std_logic_1164.all;

entity ctrl_fsm is
  port(
    clk_i   : in  std_logic;
    rst_i   : in  std_logic;
    start_i : in  std_logic;
    last_i  : in  std_logic;
    busy_o  : out std_logic;
    done_o  : out std_logic
  );
end entity ctrl_fsm;

architecture rtl of ctrl_fsm is
  type t_state is (CTRL_IDLE, CTRL_RUN, CTRL_DONE);

  signal state      : t_state;
  signal next_state : t_state;
begin
  reg_proc: process(rst_i, clk_i) is
  begin
    if rst_i = '1' then
      state <= CTRL_IDLE;
    elsif rising_edge(clk_i) then
      state <= next_state;
    end if;
  end process reg_proc;

  fsm_logic: process(state, start_i, last_i) is
  begin
    next_state <= state;
    busy_o <= '0';
    done_o <= '0';
    case state is
      when CTRL_IDLE =>
        if start_i = '1' then
          next_state <= CTRL_RUN;
        end if;
```

```
      when CTRL_RUN =>
        busy_o <= '1';
        if last_i = '1' then
          next_state <= CTRL_DONE;
        end if;
      when CTRL_DONE =>
        done_o <= '1';
        next_state <= CTRL_IDLE;
    end case;
  end process fsm_logic;
end architecture rtl;
```

In SystemVerilog:

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Moore FSM of a simple controller. A start_i pulse
// moves it from idle to run (busy_o high) until last_i, then done_o
// is high for one clock cycle before the FSM returns to idle.

`default_nettype none

module ctrl_fsm (
  input  logic clk_i,
  input  logic rst_ni,
  input  logic start_i,
  input  logic last_i,
  output logic busy_o,
  output logic done_o
);

  typedef enum logic [1:0] {
    StIdle,
    StRun,
    StDone
  } state_e;

  state_e state_d, state_q;

  // State register
  always_ff @(posedge clk_i or negedge rst_ni) begin
    if (!rst_ni) begin
      state_q <= StIdle;
    end else begin
      state_q <= state_d;
    end
```

```
  end

  // Next-state and output logic, defaults first
  always_comb begin
    state_d = state_q;
    busy_o  = 1'b0;
    done_o  = 1'b0;
    unique case (state_q)
      StIdle: begin
        if (start_i) state_d = StRun;
      end
      StRun: begin
        busy_o = 1'b1;
        if (last_i) state_d = StDone;
      end
      StDone: begin
        done_o  = 1'b1;
        state_d = StIdle;
      end
      default: state_d = StIdle;  // unused encoding 2'b11
    endcase
  end

endmodule

`default_nettype wire
```

- The state type is a `typedef enum` with an explicit base type (`logic [1:0]`). Following the lowRISC style guide, the type name ends in `_e` and the state names are UpperCamelCase with the prefix `St`.
- In VHDL, a `case` over an enumeration type with three values is complete. The 2-bit state of the SystemVerilog version has a fourth encoding, `2'b11`, which the `default` item sends back to idle.
- As in VHDL, the next state defaults to the current state, and every output defaults to its inactive value before the `case`.
- The synthesis tool is free to choose the state encoding. Yosys 0.70 recognizes this FSM and re-encodes it one-hot, with three flip-flops instead of two.
- Write the transitions with `if` and `else`. A conditional operator between enumeration values, such as `state_d = start_i ? StRun : StIdle;`, is rejected by Icarus Verilog 14, which requires an explicit cast for it.

> Exercise: The Controller as a Mealy FSM
>
> In a Mealy FSM, the outputs depend on the state and on the current inputs.

1. The combinational process of the controller reads `start_i` and `last_i`. Why is the controller still a Moore FSM?
2. Change the controller into a Mealy FSM `ctrl_fsm_mealy` with the same ports: `done_o` is high in the clock cycle in which `last_i` is high in the run state, and the FSM returns to idle at the next rising clock edge. `busy_o` stays high in the run state, as before. How many states do you need? Draw the state diagram, with the condition and the outputs on each arc (`condition / busy_o done_o`).
3. Implement `ctrl_fsm_mealy` in the two-process style in VHDL (`ctrl_fsm_mealy_ea.vhd`) and in SystemVerilog (`ctrl_fsm_mealy.sv`), and lint the SystemVerilog version.
4. Number the clock cycles from 0, with the FSM idle in cycle 0. `start_i` is high in cycle 0, `last_i` in cycle 3, and both are low otherwise. In which cycles are `busy_o` and `done_o` high in the Moore and in the Mealy version? What does the Mealy version gain, and what does its `done_o` cost?

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

Solution: The Controller as a Mealy FSM

1. The combinational process reads `start_i` and `last_i` only in the conditions that choose `state_d`. `busy_o` and `done_o` are assigned in each state without any condition, so they depend only on `state_q`. An output becomes a Mealy output as soon as it is assigned inside a condition on an input.
2. Two states are enough. `done_o` now belongs to the transition from `StRun` back to `StIdle`, in the clock cycle in which `last_i` is high, so `StDone` is no longer needed. Figure 8 shows the state diagram.

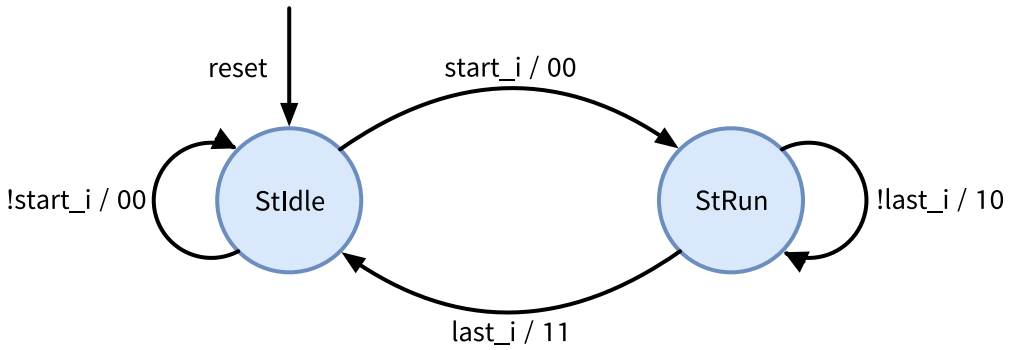


Figure 8: State diagram of the controller as a Mealy FSM. Each arc carries its condition and, after the slash, the outputs `busy_o` and `done_o` in the clock cycle in which the condition holds. `done_o` is high on the arc from `StRun` back to `StIdle`, in the same cycle as `last_i`.

3. Compared with the Moore version, only the run state changes: the assignment to `done_o` moves into the condition on `last_i`, which now returns to idle.

```vhdl
-- SPDX-FileCopyrightText: 2026 Simon Dorrer
-- SPDX-License-Identifier: Apache-2.0
-- Description: Mealy FSM of the simple controller. A start_i pulse
-- moves it from idle to run (busy_o high). done_o is high in the
-- clock cycle in which last_i is high, then the FSM returns to idle.

library ieee;
use ieee.std_logic_1164.all;

entity ctrl_fsm_mealy is
  port(
    clk_i   : in  std_logic;
    rst_i   : in  std_logic;
    start_i : in  std_logic;
    last_i  : in  std_logic;
    busy_o  : out std_logic;
    done_o  : out std_logic
  );
end entity ctrl_fsm_mealy;

architecture rtl of ctrl_fsm_mealy is
  type t_state is (CTRL_IDLE, CTRL_RUN);

  signal state      : t_state;
  signal next_state : t_state;
begin
  reg_proc: process(rst_i, clk_i) is
  begin
    if rst_i = '1' then
      state <= CTRL_IDLE;
    elsif rising_edge(clk_i) then
      state <= next_state;
    end if;
  end process reg_proc;

  fsm_logic: process(state, start_i, last_i) is
  begin
    next_state <= state;
    busy_o <= '0';
    done_o <= '0';
    case state is
      when CTRL_IDLE =>
        if start_i = '1' then
          next_state <= CTRL_RUN;
        end if;
```

```
      when CTRL_RUN =>
        busy_o <= '1';
        if last_i = '1' then
          done_o <= '1';  -- depends on an input: Mealy output
          next_state <= CTRL_IDLE;
        end if;
    end case;
  end process fsm_logic;
end architecture rtl;
```

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Mealy FSM of the simple controller. A start_i pulse
// moves it from idle to run (busy_o high). done_o is high in the
// clock cycle in which last_i is high, then the FSM returns to idle.

`default_nettype none

module ctrl_fsm_mealy (
  input  logic clk_i,
  input  logic rst_ni,
  input  logic start_i,
  input  logic last_i,
  output logic busy_o,
  output logic done_o
);

  typedef enum logic {
    StIdle,
    StRun
  } state_e;

  state_e state_d, state_q;

  // State register
  always_ff @(posedge clk_i or negedge rst_ni) begin
    if (!rst_ni) begin
      state_q <= StIdle;
    end else begin
      state_q <= state_d;
    end
  end

  // Next-state and output logic, defaults first
```

```
  always_comb begin
    state_d = state_q;
    busy_o  = 1'b0;
    done_o  = 1'b0;
    unique case (state_q)
      StIdle: begin
        if (start_i) state_d = StRun;
      end
      StRun: begin
        busy_o = 1'b1;
        if (last_i) begin
          done_o  = 1'b1;  // depends on an input: Mealy output
          state_d = StIdle;
        end
      end
      default: state_d = StIdle;
    endcase
  end

endmodule

`default_nettype wire
```

The 1-bit state type of the SystemVerilog version has no unused encoding, and the `default` item stays only because the guidelines ask for `unique case` with a `default` (Section 2.9). Verilator reports no warning. Yosys 0.70 builds the Mealy controller from 4 cells with one flip-flop, against 9 cells with three flip-flops for the one-hot Moore version.

4. Table 13 lists both versions cycle by cycle. Icarus Verilog, Verilator, and GHDL show exactly these values for the VHDL and the SystemVerilog versions.

Table 13: The Moore and the Mealy controller for the input sequence of step 4

Clock cycle	0	1	2	3	4	5
`start_i`	1	0	0	0	0	0
`last_i`	0	0	0	1	0	0
Moore state	`StIdle`	`StRun`	`StRun`	`StRun`	`StDone`	`StIdle`
Moore `busy_o`	0	1	1	1	0	0
Moore `done_o`	0	0	0	0	1	0
Mealy state	`StIdle`	`StRun`	`StRun`	`StRun`	`StIdle`	`StIdle`
Mealy `busy_o`	0	1	1	1	0	0
Mealy `done_o`	0	0	0	1	0	0

The Mealy version raises `done_o` one clock cycle earlier, in the same cycle as `last_i`, needs one state less, and is ready for the next start one cycle earlier. The price is the combinational path from `last_i` to `done_o`. The output can glitch when `last_i` changes, and the path from the register that drives `last_i`, through this FSM, into the block that reads `done_o` must fit into one clock period. If that block also drives `last_i` combinationally from `done_o`, the two blocks even form a combinational loop. A Moore output comes directly from the state register and has none of these problems. If the earlier output is not needed, a register behind the Mealy output gives back the Moore timing.

2.5.10 Hierarchy and Generate

An instance of the adder of Section 2.5.1 with a width of 16 bits looks as follows in VHDL:

```
adder_0: entity work.adder
  generic map(
    WIDTH => 16
  )
  port map(
    a_i => x,
    b_i => y,
    sum_o => s
  );
```

And in SystemVerilog:

```
adder #(
  .WIDTH(16)
) adder_0 (
  .a_i  (x),
  .b_i  (y),
  .sum_o(s)
);
```

Always connect parameters and ports by name, never by position, and connect every port. Avoid the shortcuts `.*` and `.name`, which connect ports implicitly to signals of the same name and make the code hard to review.

A `for` loop outside a process is a generate loop, which creates hardware once per iteration:

```
gen_lane: for i in 0 to 3 generate
  adder_i: entity work.adder
    generic map(
      WIDTH => 8
    )
    port map(
      a_i => lane_a(i),
      b_i => lane_b(i),
```

```
      sum_o => lane_sum(i)
    );
end generate gen_lane;
```

```
logic [3:0][7:0] lane_a, lane_b;
logic [3:0][8:0] lane_sum;

for (genvar i = 0; i < 4; i++) begin : gen_lane
  adder #(
    .WIDTH(8)
  ) adder_i (
    .a_i  (lane_a[i]),
    .b_i  (lane_b[i]),
    .sum_o(lane_sum[i])
  );
end
```

The name of the block (`gen_lane`) becomes part of the hierarchical instance names, e.g., `gen_lane[2].adder_i`. Similarly, `if (PARAM) begin : gen_name ... end` outside a process creates hardware only if the parameter is true.

2.5.11 Packages, Typedefs, and Functions

A `package` collects constants, types, and functions that several modules share, like a VHDL package:

```
package alu_pkg;

  localparam int unsigned DATA_WIDTH = 8;

  typedef enum logic [1:0] {
    OpAdd,
    OpSub,
    OpAnd,
    OpOr
  } alu_op_e;

  typedef struct packed {
    alu_op_e               op;
    logic [DATA_WIDTH-1:0] operand;
  } instr_t;

  function automatic logic is_zero(input logic [DATA_WIDTH-1:0] data);
    is_zero = (data == '0);
  endfunction

endpackage
```

A module refers to the package contents with the scope operator `::`, and the package file must be compiled before the modules that use it:

```
module alu (
  input  logic [alu_pkg::DATA_WIDTH-1:0] a_i,
  input  alu_pkg::instr_t                instr_i,
  output logic [alu_pkg::DATA_WIDTH-1:0] result_o,
  output logic                           zero_o
);

  always_comb begin
    unique case (instr_i.op)
      alu_pkg::OpAdd: result_o = a_i + instr_i.operand;
      alu_pkg::OpSub: result_o = a_i - instr_i.operand;
      alu_pkg::OpAnd: result_o = a_i & instr_i.operand;
      default:        result_o = a_i | instr_i.operand;
    endcase
  end

  assign zero_o = alu_pkg::is_zero(result_o);

endmodule
```

- A `typedef struct packed` replaces a VHDL record. Its fields are accessed with a dot (`instr_i.op`), and the whole struct is also a plain bit vector, so it can be used as a port or stored in a register.
- A `function automatic` describes combinational logic, and every call becomes a separate copy of that logic. Declare functions `automatic`, so that every call gets its own variables. Tasks, which may contain timing, belong in testbenches.

! Tool Support for SystemVerilog Features

The default SystemVerilog front end of Yosys, which LibreLane uses unless `USE_SLANG: True` is set in its configuration, supports only part of the language. In Yosys 0.70, it fails on three constructs that Verilator and Icarus Verilog accept: `import alu_pkg::*` (the counterpart of `use work.alu_pkg.all`), `return` statements in functions, and unpacked arrays as ports. An `import` inside the module or its header gives a syntax error, and one before the `module` keyword gives a syntax error or an internal error. The code of this chapter therefore uses the scope operator `pkg::NAME`, assigns a function result to the function name, and uses packed arrays at ports. The slang front end of Yosys (`read_slang`, enabled in LibreLane with `USE_SLANG: True`) accepts all of these constructs. For the same reason, the counter of the AMS chip template uses `` `define `` macros instead of a package. When you use struct fields inside `always_comb`, Icarus Verilog prints a "sorry" message that the block becomes sensitive to the whole struct, which does not change the result.

2.5.12 What Is Synthesizable

Synthesis understands only a subset of SystemVerilog. Everything else is for simulation and testbenches only (Table 14).

Table 14: The synthesizable subset of SystemVerilog

Synthesizable (RTL)	Simulation and testbenches only
`module`, `parameter`, `localparam`, `assign`	`initial` blocks (FPGA tools accept them for register initialization, ASICs have no initial values, use a reset)
`always_comb`, `always_ff`, `always_latch`	delays (`#10`), event waits inside procedural code (`@(...)`, `wait`)
`if`, `case`, `for` loops with constant bounds	system tasks such as `$display`, `$finish`, `$fatal`, `$dumpvars`
`function automatic`, generate loops and conditions	`===` and `!==`, `real`, `string`
`typedef enum`, `typedef struct packed`, packed arrays, packages	classes, `randomize()`, `fork` and `join`, tasks with timing
`+`, `-`, `*`, shifts, comparisons on vectors	loops whose number of iterations depends on data

Be careful with `/` and `%`: they synthesize to large and slow logic unless the divisor is a constant power of two, in which case they reduce to a shift or a bit selection.

2.5.13 Verilog versus SystemVerilog

SystemVerilog grew out of Verilog. Verilog became the IEEE standard 1364 in 1995, was revised in 2001 and 2005, and was merged into SystemVerilog (IEEE 1800) in 2009. SystemVerilog is therefore a superset of Verilog: a Verilog file is also valid SystemVerilog, unless it uses one of the new keywords, such as `logic` or `bit`, as a name. In this course, you write SystemVerilog, but you will read plain Verilog all the time:

- **Netlists.** Yosys and LibreLane write their netlists in plain Verilog, e.g., `final/nl/counter_top.nl.v` of the counter macro in the AMS chip template: a port list in the style of Verilog-1995, a long list of `wire`s, and instances of standard cells such as `sg13cmos5l_dfrbpq_1`.
- **Cell models.** A gate-level simulation needs a simulation model of every standard cell. The models in the PDK (`sg13cmos5l_stdcell.v`, with the user-defined primitives in `sg13cmos5l_udp.v`) are written in Verilog with gate primitives, user-defined primitives, and `specify` blocks for the timing.
- **Generated code.** All alternative HDLs of Section 2.3 emit Verilog or SystemVerilog.
- **Existing code.** Many open-source cores, older IP blocks, textbooks, and online examples are written in Verilog-2001, as is the Verilog cheat-sheet of Section 2.4. AI assistants often suggest code in the same style.

Table 15 maps the Verilog idioms you will meet onto the SystemVerilog of this chapter.

Table 15: Plain Verilog versus SystemVerilog

Plain Verilog	SystemVerilog	Remark
`module m (a, y);` followed by `input a;` and `output y;`	`module m (input logic a_i, output logic y_o);`	Verilog-1995 style, still used in netlists
`wire` for `assign` and instance outputs, `reg` for `always` blocks	`logic` for both	the Verilog type depends on how a signal is assigned
`always @(a or b)`, since Verilog-2001 `always @(*)`	`always_comb`	complete sensitivity, latch check (Section 2.5.8)
`always @(posedge clk or negedge rst_n)`	`always_ff @(posedge clk_i or negedge rst_ni)`	states the intent: flip-flops
`always @(*)` that misses an assignment	`always_latch`	documents an intended latch
`parameter W = 8`, `` `define ``	`parameter int unsigned W = 8`, `package`	typed constants, packages instead of global macros
a `localparam` or `` `define `` per state	`typedef enum logic [1:0] {...} state_e`	assigning a plain number needs a cast (an error in Verilator and Icarus Verilog)
slices of a wide vector, e.g., `instr[9:8]`	`typedef struct packed`	named fields, e.g., `instr.op`
`{W{1'b0}}`	`'0`	fills any width
`integer i;` in the module, `i = i + 1`	`for (int i = 0; i < N; i++)`	a loop variable local to the loop
`case` with `// synopsys full_case parallel_case`	`unique case` with `default`	Yosys and Verilator read the comment, Icarus Verilog ignores it
`.v`, `read_verilog`, `iverilog`	`.sv`, `read_verilog -sv`, `iverilog -g2012`	Verilator reads both as SystemVerilog

Most of these differences are syntax, but three of them matter beyond syntax:

- **`reg` and `wire` follow the assignment, not the hardware.** In Verilog, a signal assigned in an `always` block must be a `reg`, even if it is combinational, and a signal driven by an `assign` or by an instance must be a `wire`. If you turn an `assign` into an `always` block, you therefore have to change the declaration as well. In SystemVerilog, a variable (`logic` or `reg`) may be assigned in an `always` block or by a single `assign`, so the declaration stays the same, and no type name suggests a register that does not exist. Only a `wire` assigned in an `always`

block remains an error (Icarus Verilog: not a valid l-value, Verilator: `PROCASSWIRE` for an internal `wire`).

- **The process types state the intent.** For a missing assignment in `always @(*)`, Verilator and Yosys 0.70 only warn, and Yosys builds the latch. For `always_comb`, Yosys stops with an error (Section 2.5.8).
- **Checks belong in the language, not in comments.** In Verilog, the comment `// synopsys full_case parallel_case` after a `case` tells the synthesis tool that all missing values are don't care (`full_case`) and that the items never overlap (`parallel_case`). Yosys reads it and builds no latch, and Verilator reads it as well: it drops the `LATCH` and `CASEINCOMPLETE` warnings and stops the simulation when no item matches. Icarus Verilog ignores the comment and keeps the old value, so its simulation can differ from the netlist. Write a complete `case` with a `default` item instead, and add `unique` if the items are mutually exclusive. Keep the `default` with `unique` as well. Without it, Yosys handles `unique` like `full_case`: it silently takes the missing values as don't care, even in `always_comb`, and the netlist can differ from the simulation.

Most testbench features of this chapter are SystemVerilog as well, e.g., `$fatal`, `$isunknown()`, and `int` variables, and the verification part of the language (classes, randomization, assertions, and coverage) has no Verilog counterpart.

Finally, the tools must know which language they read. Verilator reads every file as SystemVerilog, also a `.v` file. Icarus Verilog reads Verilog-2005 unless you select SystemVerilog with `-g2012`, and the Yosys command `read_verilog` needs the option `-sv`, even for a file that ends in `.sv`. Name your files `.sv`, so that readers and editors know which language is inside.

Exercise: Modernize a Verilog-2001 Module

The following priority encoder with registered outputs is written in plain Verilog-2001, as you find it in many older designs: `idx` is the index of the lowest set bit of `req`, and `valid` is high if any bit of `req` is set. Besides its old style, the module contains one mistake.

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Priority encoder with registered outputs in plain
// Verilog-2001. idx is the index of the lowest set bit of req,
// valid is high if any bit of req is set. This file contains a
// mistake on purpose (exercise), do not use it as a template.

module prio_enc (clk, rst_n, req, idx, valid);

  parameter N = 8;
  parameter W = 3;  // width of idx, must be log2(N)

  input          clk;
  input          rst_n;
  input  [N-1:0] req;
  output [W-1:0] idx;
```

```
  output          valid;

  reg    [W-1:0] idx;
  reg            valid;
  reg    [W-1:0] idx_next;
  wire           valid_next;
  integer        i;

  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      idx   <= {W{1'b0}};
      valid <= 1'b0;
    end else begin
      idx   <= idx_next;
      valid <= valid_next;
    end
  end

  assign valid_next = |req;

  // The loop runs from the highest to the lowest bit, so the
  // lowest set bit is assigned last and wins.
  always @(*) begin
    for (i = N - 1; i >= 0; i = i - 1)
      if (req[i])
        idx_next = i;
  end

endmodule
```

1. Which constructs are plain Verilog, and what are their SystemVerilog counterparts in Table 15?
2. Save the file as `prio_enc.v`, lint it with `verilator --lint-only -Wall prio_enc.v`, and synthesize it with `yosys -p "read_verilog prio_enc.v; synth -top prio_enc; stat"`. What do the two tools report?
3. Translate the module into SystemVerilog in `prio_enc.sv`, with the conventions of this chapter: ANSI ports with `_i` and `_o`, `rst_ni`, `logic`, and the two-process style with `_d` and `_q`. Do not fix the mistake yet. Synthesize your translation with `read_verilog -sv`. What has changed?
4. Fix the mistake and the remaining lint warning, and lint and synthesize the module again.

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

Solution: Modernize a Verilog-2001 Module

1. Each plain Verilog construct and its SystemVerilog counterpart:
   - The port list with separate declarations (Verilog-1995 style) becomes an ANSI header with `logic` ports.
   - The `reg`s (the outputs and `idx_next`, assigned in `always` blocks) and the `wire valid_next` (driven by an `assign`) all become `logic`.
   - `always @(*)` and `always @(posedge clk or negedge rst_n)` become `always_comb` and `always_ff`.
   - The module-level `integer i` with `i = i - 1` becomes `for (int i = N - 1; i >= 0; i--)`.
   - The replication `{W{1'b0}}` becomes `'0`.
   - The untyped parameters become `parameter int unsigned`, and `W`, which must be set to match `N` by hand, becomes `$clog2(N)`.
   - The file extension `.v` becomes `.sv`.
2. Verilator 5.052 reports two warnings. `WIDTHTRUNC`: the 32-bit `integer i` is assigned to the 3-bit `idx_next`. `LATCH`: if no bit of `req` is set, the loop assigns nothing, and `idx_next` keeps its old value. Yosys 0.70 only prints the warning `Latch inferred for signal` `` `\prio_enc.\idx_next' `` and builds 23 cells, among them four flip-flops `$_DFF_PN0_` for `idx` and `valid` and three latches `$_DLATCH_N_` for `idx_next`. Leaving a value unassigned does not mean "don't care" but "keep the old value."
3. With `always_comb`, Yosys stops with `ERROR: Latch inferred for signal` `` `\prio_enc.\idx_d' `` `from always_comb process`. Verilator reports the same `LATCH` warning as before, and `WIDTHTRUNC` again unless your translation already casts `i`.
4. The fix is a default assignment, as in every combinational process of this chapter:

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Priority encoder with registered outputs. idx_o
// is the index of the lowest set bit of req_i, valid_o is high if
// any bit of req_i is set.

`default_nettype none

module prio_enc #(
  parameter int unsigned N = 8
) (
  input  logic                clk_i,
  input  logic                rst_ni,
  input  logic [N-1:0]        req_i,
  output logic [$clog2(N)-1:0] idx_o,
  output logic                valid_o
);

  localparam int unsigned W = $clog2(N);
```

```
  logic [W-1:0] idx_d, idx_q;
  logic         valid_d, valid_q;

  // Register process: reset values and copies of the _d signals only
  always_ff @(posedge clk_i or negedge rst_ni) begin
    if (!rst_ni) begin
      idx_q   <= '0;
      valid_q <= 1'b0;
    end else begin
      idx_q   <= idx_d;
      valid_q <= valid_d;
    end
  end

  // Combinational process: defaults first, then all decisions
  always_comb begin
    idx_d   = '0;
    valid_d = |req_i;
    // The loop runs from the highest to the lowest bit, so the
    // lowest set bit is assigned last and wins.
    for (int i = N - 1; i >= 0; i--) begin
      if (req_i[i]) begin
        idx_d = W'(i);
      end
    end
  end

  assign idx_o   = idx_q;
  assign valid_o = valid_q;

endmodule

`default_nettype wire
```

- The width `W` follows from `N` with `$clog2` (available since Verilog-2005), so it can no longer be set wrong.
- The size cast `W'(i)` makes the truncation of the loop variable explicit.
- The loop variable is declared in the loop. It must be a signed `int`: with `int unsigned`, the condition `i >= 0` is always true. Verilator 5.052 reports a constant comparison (`UNSIGNED`) and an infinite loop (`INFINITELOOP`) and then stops with an internal error.

Verilator reports no warning, and Yosys builds 22 cells with the four flip-flops and no latch. While `valid_o` is low, `idx_o` is now 0, where the original kept the last index (or `x` before the first request). This does not matter, since `idx_o` is only meaningful while

`valid_o` is high. The default `idx_d = idx_q;` would also be correct: it keeps the last index in the register, and Yosys turns the three `idx` flip-flops into flip-flops with enable (`$_DFFE_PN0P_`), again without a latch.

The testbench `prio_enc/testbenches/verilog/prio_enc_tb.sv` in the code folder applies all 256 values of `req` and compares the outputs with a reference model, checking `idx_o` only while `valid_o` is high. It also runs with your own translation, as long as it has the parameter `N` and the ports `clk_i`, `rst_ni`, `req_i`, `idx_o`, and `valid_o`:

```
iverilog -g2012 -o prio_enc_tb.vvp prio_enc.sv prio_enc_tb.sv
vvp prio_enc_tb.vvp
```

2.6 Testbenches

A testbench applies stimuli to the design under test (DUT) and checks its responses. It is simulation-only code, so it may use the whole language. Make every testbench **self-checking**: it compares the outputs with the expected values and reports a clear pass or fail, so that you never have to judge a waveform by eye to know whether a design works. Waveforms are for debugging. This section shows a SystemVerilog and a cocotb testbench for the tick generator of Section 2.5.7. Section 4 covers verification with cocotb in depth.

2.6.1 A SystemVerilog Testbench

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Self-checking SystemVerilog testbench for tick_gen.

`timescale 1ns / 1ps

module tick_gen_tb;

  localparam int unsigned DIV          = 5;
  localparam real         CLK_PERIOD_NS = 20.0;  // 50 MHz

  logic clk   = 1'b0;
  logic rst_n = 1'b0;  // reset is active from time 0
  logic en    = 1'b0;
  logic tick;

  // Device under test (DUT)
  tick_gen #(
    .DIV(DIV)
  ) dut (
    .clk_i (clk),
    .rst_ni(rst_n),
    .en_i  (en),
    .tick_o(tick)
```

```
  );

  // Clock generation
  always #(CLK_PERIOD_NS / 2) clk = ~clk;

  // Stimulus and checks: inputs change and outputs are sampled on the
  // falling edge, so they never race with the DUT at the rising edge.
  initial begin
    $dumpfile("tick_gen_tb.vcd");
    $dumpvars(0, tick_gen_tb);

    repeat (2) @(negedge clk);
    rst_n = 1'b1;

    // Disabled: no tick at all
    repeat (3 * DIV) begin
      @(negedge clk);
      if (tick !== 1'b0) $fatal(1, "FAIL: tick while disabled");
    end

    // Enabled: exactly one tick every DIV clock cycles
    en = 1'b1;
    for (int cycle = 1; cycle <= 4 * DIV; cycle++) begin
      @(negedge clk);
      if (tick !== (cycle % DIV == 0)) begin
        $fatal(1, "FAIL: cycle %0d, tick = %b", cycle, tick);
      end
    end

    $display("PASS: tick_gen_tb");
    $finish;
  end

endmodule
```

- `` `timescale 1ns / 1ps `` sets the unit of the delays (#) and the precision of the simulation.
- The testbench is a module without ports. It instantiates the DUT with a smaller `DIV` than the default, which keeps the simulation short.
- `always #(CLK_PERIOD_NS / 2) clk = ~clk;` generates the clock. Initial values in declarations are fine in a testbench.
- The stimulus runs in an `initial` block. **Inputs are changed and outputs are checked on the falling clock edge** (`@(negedge clk)`), while the DUT acts on the rising edge. If the testbench and the DUT used the same edge, the result would depend on the order in which the simulator runs the processes, and Icarus Verilog and Verilator choose this order differently.

- The checks use `!==`, which also fails if the output is `x`, and end the simulation with `$fatal(1, ...)` on the first error. Do not use `$error` for this: Icarus Verilog only prints the message, reaches `$finish`, and exits with status 0, so a script or a make target cannot see that the test failed.
- `$dumpfile` and `$dumpvars` write all signals into a VCD waveform file, and `$finish` ends the simulation.

2.6.2 A cocotb Testbench

cocotb [10] runs the testbench as Python code next to the simulator. Python then drives and reads the signals of the DUT through the programming interface of the simulator (the Verilog Procedural Interface, VPI, for Verilog simulators).

```
# SPDX-FileCopyrightText: 2026 Simon Dorrer
# SPDX-License-Identifier: Apache-2.0
# Description: Self-checking cocotb testbench for tick_gen.

import os
from pathlib import Path

import cocotb
from cocotb.clock import Clock
from cocotb.triggers import ClockCycles, FallingEdge
from cocotb_tools.runner import get_results, get_runner

DIV = 5  # passed to the DUT as parameter in the runner below


async def start_up(dut):
    """Start a 50 MHz clock and reset the DUT for two cycles."""
    cocotb.start_soon(Clock(dut.clk_i, 20, "ns").start())
    dut.rst_ni.value = 0
    dut.en_i.value = 0
    await ClockCycles(dut.clk_i, 2)
    await FallingEdge(dut.clk_i)
    dut.rst_ni.value = 1


@cocotb.test()
async def test_no_tick_when_disabled(dut):
    """tick_o stays low while en_i is low."""
    await start_up(dut)
    for _ in range(3 * DIV):
        await FallingEdge(dut.clk_i)
        assert dut.tick_o.value == 0, "tick while disabled"


@cocotb.test()
```

```
async def test_tick_every_div_cycles(dut):
    """tick_o is high for one cycle every DIV cycles."""
    await start_up(dut)
    dut.en_i.value = 1
    for cycle in range(1, 4 * DIV + 1):
        await FallingEdge(dut.clk_i)
        expected = 1 if cycle % DIV == 0 else 0
        assert dut.tick_o.value == expected, \
            f"cycle {cycle}: expected {expected}, got {dut.tick_o.value}"


def tick_gen_runner():
    sim = os.getenv("SIM", "icarus")
    proj_path = Path(__file__).resolve().parent

    runner = get_runner(sim)
    runner.build(
        sources=[proj_path / "../../rtl/tick_gen.sv"],
        hdl_toplevel="tick_gen",
        parameters={"DIV": DIV},
        always=True,
        waves=True,
        timescale=("1ns", "1ps"),
    )
    results_xml = runner.test(
        hdl_toplevel="tick_gen",
        test_module="tick_gen_tb",
        waves=True,
    )

    # A failing test leaves the exit status at 0, so check the results
    num_tests, num_failed = get_results(results_xml)
    if num_failed:
        raise SystemExit(
            f"FAIL: {num_failed} of {num_tests} tests failed")


if __name__ == "__main__":
    tick_gen_runner()
```

- Every function decorated with `@cocotb.test()` is a test. The tests of a file run one after another in the same simulation, so each one starts its own clock and applies the reset.
- Tests are coroutines (`async def`), which wait for simulator events with `await`: `RisingEdge(sig)`, `FallingEdge(sig)`, `ClockCycles(sig, n)`, or `Timer(t, "ns")`. As in the SystemVerilog testbench, inputs are driven and outputs are checked on the falling edge.

- `dut.en_i.value = 1` writes an input, and `dut.tick_o.value` reads an output. A write takes effect at the end of the current time step, so reading the same signal right after writing it still returns the old value.
- `cocotb.start_soon(Clock(dut.clk_i, 20, "ns").start())` starts a 50 MHz clock that runs in parallel with the test.
- A failing Python `assert` fails the test, and the f-string message explains why.
- The function `tick_gen_runner()` follows the idiom of the AMS chip template: the file runs itself. It selects the simulator from the environment variable `SIM` (Icarus Verilog by default), builds the DUT with the given parameters, and runs the tests. `runner.test()` returns the path of the result file `results.xml`, but a failing test does not change the exit status of `python3`. The last lines therefore check the results with `get_results()` and exit with an error if any test failed.

2.6.3 SystemVerilog or cocotb?

Table 16 compares the two kinds of testbenches.

Table 16: SystemVerilog and cocotb testbenches

Aspect	SystemVerilog testbench	cocotb testbench
Language	SystemVerilog (the full language, including the simulation subset)	Python 3
Reference models	written in SystemVerilog	any Python code and library (e.g., NumPy)
Simulators	every SystemVerilog simulator	Icarus Verilog, Verilator, GHDL, and commercial simulators
DUT languages	SystemVerilog, VHDL only with a mixed-language simulator	the same testbench works for VHDL and SystemVerilog DUTs
Speed	fast, everything runs in the simulator	slower, every interaction passes through Python
In the AMS chip template	`testbenches/verilog/`	`testbenches/cocotb/`

In this course, we use both: a SystemVerilog testbench for a quick self-check of a block, and cocotb for everything that needs a reference model or randomized stimuli.

2.7 Linting, Simulation, and Synthesis in the IIC-OSIC-TOOLS Container

All tools you need are installed in the IIC-OSIC-TOOLS container (Section 1.9). In the AMS chip template, they are wrapped in make targets. In this chapter, you type the commands yourself, so that you know what is behind the targets when you use them later (Section 2.8).

2.7.1 Setting Up a Working Folder

Start the container (Section 1.13) and open a terminal on its desktop. Create a folder for the tick generator with the same structure as a macro of the AMS chip template:

```
mkdir -p /foss/designs/sv_exercises/tick_gen/{rtl,testbenches/verilog,testbenches/cocotb}
cd /foss/designs/sv_exercises/tick_gen
```

Save the three listings as `rtl/tick_gen.sv`, `testbenches/verilog/tick_gen_tb.sv`, and `testbenches/cocotb/tick_gen_tb.py`, e.g., with `gedit`, `vim`, or VS Code. With a local installation, `/foss/designs` is the folder `~/eda/designs` on your laptop, so your files survive when the container is removed or updated. All commands of this section start in the folder `/foss/designs/sv_exercises/tick_gen`.

2.7.2 Linting with Verilator

A linter checks the code for errors and suspicious constructs without simulating it. Run it first, on every RTL file, after every change:

```
verilator --lint-only -Wall rtl/tick_gen.sv
```

`--lint-only` only parses and checks the code, and `-Wall` enables all warnings, including the style warnings. Without any warning, Verilator only prints a short report and exits with status 0. Otherwise, it prints the warnings with file, line, and a link to their description, and exits with an error. Table 17 lists the warnings you will meet most often.

Table 17: Frequent Verilator warnings

Warning	Meaning	Typical fix
`IMPLICIT`	an undeclared name, Verilog created a 1-bit wire	fix the typo, add `` `default_nettype none ``
`UNDRIVEN`, `UNUSEDSIGNAL`	a signal is never assigned or never read	connect it or remove it
`WIDTHTRUNC`, `WIDTHEXPAND`	the widths of operands or of an assignment do not match	size the literals, extend with `{1'b0, x}`, or cast with `N'(x)`
`LATCH`	a combinational block (`always_comb` or `always @(*)`) does not assign a variable on every path	default assignments first, complete `case` and `else`
`CASEINCOMPLETE`	a `case` does not cover all values	add the missing items or a `default`
`BLKSEQ`	a blocking assignment `=` in `always_ff`	use `<=` instead
`MULTIDRIVEN`	a variable is driven from more than one block	assign it in exactly one block
`DECLFILENAME`	the file name differs from the module name	one module per file, with the same name

If a warning is a false alarm, you can switch it off for a few lines with `/* verilator lint_off WIDTHEXPAND */` and `/* verilator lint_on WIDTHEXPAND */`, together with a comment that explains why. Never switch off warnings globally.

2.7.3 Simulating a SystemVerilog Testbench

Icarus Verilog compiles the design and the testbench into a simulation file, which `vvp` then runs:

```
iverilog -g2012 -o testbenches/verilog/tick_gen_tb.vvp rtl/tick_gen.sv
testbenches/verilog/tick_gen_tb.sv
cd testbenches/verilog
vvp tick_gen_tb.vvp
cd ../..
```

`-g2012` selects the SystemVerilog standard IEEE 1800-2012, and `-o` names the output file. List the files in compile order: packages and files with `` `define `` macros first, then the modules. `vvp` writes the VCD file into the current folder, which is why we change into the testbench folder first and return afterwards. The simulation prints `PASS: tick_gen_tb` and exits with status 0, or stops at the first failed check with `FATAL` and exit status 1 (check it with `echo $?`).

Verilator can also run SystemVerilog testbenches. It compiles them into a C++ program, which is much faster for large designs. `--binary` builds an executable, and `--timing` enables the delays and event controls of the testbench. `--timescale` gives the RTL, which has no `` `timescale `` of its own, the same time unit as the testbench, and `--trace` enables the VCD output:

```
verilator --binary --timing --timescale 1ns/1ps --trace --top-
module tick_gen_tb -o tick_gen_tb rtl/tick_gen.sv testbenches/verilog/
tick_gen_tb.sv
cd testbenches/verilog
../../obj_dir/tick_gen_tb
cd ../..
```

2.7.4 Running a cocotb Testbench

A cocotb testbench in the template idiom runs itself:

```
cd testbenches/cocotb
python3 tick_gen_tb.py                  # with Icarus Verilog (default)
SIM=verilator python3 tick_gen_tb.py    # with Verilator
cd ../..
```

cocotb builds the simulation in the folder `sim_build/`, runs all tests, and prints a summary table with one line per test and a final line such as `TESTS=2 PASS=2 FAIL=0 SKIP=0`. The results are also stored in `sim_build/results.xml`. With Icarus Verilog, the waveform is written to `sim_build/tick_gen.fst`, with Verilator to `sim_build/dump.vcd`.

2.7.5 Viewing Waveforms

Open a waveform with GTKWave or Surfer:

```
gtkwave testbenches/verilog/tick_gen_tb.vcd
surfer testbenches/cocotb/sim_build/tick_gen.fst
```

In GTKWave, select a module in the hierarchy on the left, append its signals to the wave window, and zoom to fit. GTKWave can store the selected signals in a save file (`.gtkw`), which you pass as a second argument the next time, e.g., `gtkwave tick_gen_tb.vcd tick_gen_tb.gtkw` in the testbench folder. The AMS chip template keeps such save files next to its testbenches.

2.7.6 A Quick Synthesis Check with Yosys

A simulation shows *what* a design does, but not *which hardware* it becomes. A quick synthesis with Yosys answers this in a second:

```
yosys -p "read_verilog -sv rtl/tick_gen.sv; synth -top tick_gen; stat"
```

`read_verilog -sv` reads the file with the SystemVerilog front end, `synth -top tick_gen` synthesizes the design with `tick_gen` as top module onto generic gates, and `stat` prints the cell statistics at the end of the log. Check that the number of flip-flops (`$_DFF_*` and `$_DFFE_*` cells) matches the registers you intended and that there are no latches (`$_DLATCH_*` cells). The letters of a flip-flop name give the clock edge, the reset polarity, the reset value, and the enable polarity: `$_DFFE_PN0P_` is a flip-flop on the positive clock edge (`P`) with an active-low reset (`N`) to zero (`0`) and an active-high enable (`P`). In the LibreLane flow, Yosys maps these generic cells onto the standard cells of SG13CMOS5L (Section 6).

> Exercise: Lint, Simulate, and Synthesize the Tick Generator

1. Set up the folder as described in Section 2.7.1 and save the three listings of the tick generator.
2. Lint the RTL with Verilator.
3. Run the SystemVerilog testbench with Icarus Verilog and look at the waveform in GTKWave. Find the cycles in which `tick` is high and compare them with `count_q` inside the DUT.
4. Run the cocotb testbench with Icarus Verilog and with Verilator, and open the cocotb waveform in Surfer.
5. Synthesize the RTL with Yosys. How many cells and flip-flops does it need, and of which types are the flip-flops?
6. Change `DIV` to 8 in both testbenches and rerun them. Since 8 is a power of two, the 3-bit counter uses all eight of its values. Then build in a bug: compare `count_q` with `COUNT_LAST - 1` instead of `COUNT_LAST`. Do both testbenches catch it?

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

> Solution: Lint, Simulate, and Synthesize the Tick Generator

2. Verilator reports no warning.
3. The simulation prints `PASS: tick_gen_tb`. With `DIV = 5`, `tick` is high in the clock cycle after `count_q` has reached 4, i.e., once every five cycles while `en` is high.
4. Both runs end with `TESTS=2 PASS=2 FAIL=0 SKIP=0`. Icarus Verilog writes `sim_build/tick_gen.fst`, Verilator `sim_build/dump.vcd`.

5. With the default `DIV = 10`, Yosys 0.70 reports 17 cells, among them five flip-flops: four `$_DFFE_PN0P_` cells for `count_q` (with an active-high enable input, which comes from `en_i`) and one `$_DFF_PN0_` cell for `tick_q`.
6. With `DIV = 8`, both testbenches pass. With the bug, the counter wraps one cycle early, and both testbenches fail: the SystemVerilog testbench stops with `FATAL` and exit status 1, cocotb reports `FAIL` for `test_tick_every_div_cycles`, and the runner exits with an error.

Exercise: PWM Generator with a SystemVerilog Testbench

Design a pulse-width modulation (PWM) generator `pwm` in the two-process style with these properties:

- A parameter `WIDTH` (default 8) and the ports `clk_i`, `rst_ni` (asynchronous, active low), `duty_i` (`WIDTH` bits), and `pwm_o`.
- An internal counter counts from 0 to $2^{\mathrm{WIDTH}} - 1$ and wraps around, so one PWM period lasts 2^{WIDTH} clock cycles.
- In every period, `pwm_o` is high for exactly `duty_i` clock cycles. `duty_i = 0` keeps the output low, and the maximum duty cycle is $(2^{\mathrm{WIDTH}} - 1)/2^{\mathrm{WIDTH}}$.
- `pwm_o` comes directly from a flip-flop, so that it has no glitches on its way to a pad.

1. Write the RTL in `pwm/rtl/pwm.sv` and lint it.
2. Write a self-checking SystemVerilog testbench `pwm/testbenches/verilog/pwm_tb.sv` with `WIDTH = 4`. For every one of the 16 duty values, it counts the high cycles of `pwm_o` within one period and stops with `$fatal` if the count is wrong. Also check that `pwm_o` is never `x`.
3. Run the testbench with Icarus Verilog and inspect the waveform.

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

Solution: PWM Generator with a SystemVerilog Testbench

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: PWM generator. In every period of 2**WIDTH clock
// cycles, pwm_o is high for exactly duty_i cycles (duty_i = 0 keeps
// it low).

`default_nettype none

module pwm #(
  parameter int unsigned WIDTH = 8
) (
  input  logic             clk_i,
```

```
  input  logic             rst_ni,
  input  logic [WIDTH-1:0] duty_i,
  output logic             pwm_o
);

  logic [WIDTH-1:0] count_d, count_q;
  logic             pwm_d, pwm_q;

  always_ff @(posedge clk_i or negedge rst_ni) begin
    if (!rst_ni) begin
      count_q <= '0;
      pwm_q   <= 1'b0;
    end else begin
      count_q <= count_d;
      pwm_q   <= pwm_d;
    end
  end

  always_comb begin
    count_d = count_q + 1'b1;  // wraps from 2**WIDTH - 1 to 0
    pwm_d   = (count_q < duty_i);
  end

  // Registered output: no glitches on the way to a pad
  assign pwm_o = pwm_q;

endmodule

`default_nettype wire
```

Since the counter has exactly `WIDTH` bits, it wraps around without any compare logic. The output register delays `pwm_o` by one cycle with respect to the comparison, which does not change the number of high cycles per period.

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Self-checking SystemVerilog testbench for pwm,
// testing all duty values of a 4-bit PWM.

`timescale 1ns / 1ps

module pwm_tb;

  localparam int unsigned WIDTH         = 4;
```

```
  localparam int unsigned PERIOD        = 2 ** WIDTH;
  localparam real         CLK_PERIOD_NS = 20.0;  // 50 MHz

  logic             clk   = 1'b0;
  logic             rst_n = 1'b0;
  logic [WIDTH-1:0] duty  = '0;
  logic             pwm_out;
  int unsigned      high_cycles;

  pwm #(
    .WIDTH(WIDTH)
  ) dut (
    .clk_i (clk),
    .rst_ni(rst_n),
    .duty_i(duty),
    .pwm_o (pwm_out)
  );

  always #(CLK_PERIOD_NS / 2) clk = ~clk;

  initial begin
    $dumpfile("pwm_tb.vcd");
    $dumpvars(0, pwm_tb);

    repeat (2) @(negedge clk);
    rst_n = 1'b1;

    for (int d = 0; d < PERIOD; d++) begin
      duty = WIDTH'(d);
      repeat (2) @(negedge clk);  // the new duty reaches pwm_o

      // Any PERIOD consecutive cycles contain exactly d high cycles
      high_cycles = 0;
      repeat (PERIOD) begin
        @(negedge clk);
        if ($isunknown(pwm_out)) $fatal(1, "FAIL: pwm_o is X or Z");
        if (pwm_out) high_cycles++;
      end
      if (high_cycles != d) begin
        $fatal(1, "FAIL: duty %0d gives %0d high cycles", d,
               high_cycles);
      end
    end

    $display("PASS: pwm_tb");
```

```
    $finish;
  end

endmodule
```

After a new duty value is applied, the testbench waits two cycles until it has reached the output register. Then any 2^{WIDTH} consecutive cycles contain exactly `duty` high cycles, regardless of where the period starts, so the testbench does not need to know the internal counter. `$isunknown()` is true if any bit is `x` or `z`. The commands are:

```
cd /foss/designs/sv_exercises/pwm
verilator --lint-only -Wall rtl/pwm.sv
iverilog -g2012 -o testbenches/verilog/pwm_tb.vvp rtl/pwm.sv testbenches/
verilog/pwm_tb.sv
cd testbenches/verilog && vvp pwm_tb.vvp
gtkwave pwm_tb.vcd
```

To check that the testbench really checks something, change `<` to `<=` in the comparison: the testbench then stops with `FATAL` and the message `FAIL: duty 0 gives 1 high cycles`.

Exercise: PWM Generator with a cocotb Testbench

Write a cocotb testbench `pwm/testbenches/cocotb/pwm_tb.py` for your PWM generator from the previous exercise, again with `WIDTH = 4`:

1. A test `test_all_duty_values` with the same check as the SystemVerilog testbench. In cocotb, `dut.pwm_o.value.is_resolvable` is false if any bit is `x` or `z`, so `not dut.pwm_o.value.is_resolvable` corresponds to `$isunknown()`.
2. A test `test_against_model` that changes `duty_i` to random values at random times (use a fixed seed, e.g., `random.Random(42)`) and compares `pwm_o` in every clock cycle with a cycle-accurate Python model of the counter and the output register.
3. A runner in the template idiom of Section 2.6.2, which exits with an error if a test fails.

Run the testbench with Icarus Verilog and with Verilator.

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

Solution: PWM Generator with a cocotb Testbench

```
# SPDX-FileCopyrightText: 2026 Simon Dorrer
# SPDX-License-Identifier: Apache-2.0
# Description: Self-checking cocotb testbench for pwm (4-bit PWM).
```

```
import os
import random
from pathlib import Path

import cocotb
from cocotb.clock import Clock
from cocotb.triggers import ClockCycles, FallingEdge
from cocotb_tools.runner import get_results, get_runner

WIDTH = 4
PERIOD = 2**WIDTH


async def start_up(dut):
    """Start a 50 MHz clock and reset the DUT for two cycles."""
    cocotb.start_soon(Clock(dut.clk_i, 20, "ns").start())
    dut.rst_ni.value = 0
    dut.duty_i.value = 0
    await ClockCycles(dut.clk_i, 2)
    await FallingEdge(dut.clk_i)
    dut.rst_ni.value = 1


@cocotb.test()
async def test_all_duty_values(dut):
    """For every duty d, pwm_o is high for d cycles per period."""
    await start_up(dut)
    for duty in range(PERIOD):
        dut.duty_i.value = duty
        for _ in range(2):  # the new duty reaches pwm_o
            await FallingEdge(dut.clk_i)
        high_cycles = 0
        for _ in range(PERIOD):
            await FallingEdge(dut.clk_i)
            assert dut.pwm_o.value.is_resolvable, "pwm_o is X or Z"
            high_cycles += int(dut.pwm_o.value)
        assert high_cycles == duty, \
            f"duty {duty} gives {high_cycles} high cycles"


@cocotb.test()
async def test_against_model(dut):
    """Random duty changes, checked cycle by cycle against a model."""
    await start_up(dut)
```

```
    rng = random.Random(42)  # fixed seed: every run is reproducible
    count, pwm, duty = 0, 0, 0  # model state after reset
    for _ in range(20 * PERIOD):
        if rng.random() < 0.05:
            # Keep the value in Python: a write becomes visible in
            # the DUT only at the end of the current time step
            duty = rng.randrange(PERIOD)
            dut.duty_i.value = duty
        await FallingEdge(dut.clk_i)
        # Model of the rising edge: both registers update at once
        pwm = 1 if count < duty else 0
        count = (count + 1) % PERIOD
        assert dut.pwm_o.value == pwm, \
            f"expected {pwm}, got {dut.pwm_o.value}"


def pwm_runner():
    sim = os.getenv("SIM", "icarus")
    proj_path = Path(__file__).resolve().parent

    runner = get_runner(sim)
    runner.build(
        sources=[proj_path / "../../rtl/pwm.sv"],
        hdl_toplevel="pwm",
        parameters={"WIDTH": WIDTH},
        always=True,
        waves=True,
        timescale=("1ns", "1ps"),
    )
    results_xml = runner.test(
        hdl_toplevel="pwm",
        test_module="pwm_tb",
        waves=True,
    )

    # A failing test leaves the exit status at 0, so check the results
    num_tests, num_failed = get_results(results_xml)
    if num_failed:
        raise SystemExit(
            f"FAIL: {num_failed} of {num_tests} tests failed")


if __name__ == "__main__":
    pwm_runner()
```

The model in `test_against_model` mirrors the two registers of the DUT: at every rising edge, the new `pwm` value is computed from the *old* counter value, and the counter increments, exactly like the two non-blocking assignments in `always_ff`. The comment on `duty` explains why the model keeps its own copy of the value it writes. Reading `dut.duty_i.value` right after the write would still return the old value, and the model would be one cycle off. Both simulators report `TESTS=2 PASS=2 FAIL=0 SKIP=0`. The model test is the stronger one: a counter that wraps at 14 instead of 15 fails both tests, but only the model test would also find a bug that keeps the number of high cycles right and shifts the pulse within the period.

Exercise: Sequence Detector FSM

The following VHDL FSM detects the bit pattern `1011` in a serial input. It samples one bit at each rising clock edge, and overlapping occurrences count, so `1011011` contains the pattern twice. `detect_o` is high for one clock cycle after the edge that sampled the last `1` of the pattern (Moore output).

```vhdl
-- SPDX-FileCopyrightText: 2026 Simon Dorrer
-- SPDX-License-Identifier: Apache-2.0
-- Description: Moore FSM that detects the bit pattern 1011 in the
-- serial input bit_i (one bit per rising clock edge, overlapping
-- occurrences count). detect_o is high for one cycle after the edge
-- that sampled the last 1.

library ieee;
use ieee.std_logic_1164.all;

entity seq_detect is
  port(
    clk_i    : in  std_logic;
    rst_i    : in  std_logic;
    bit_i    : in  std_logic;
    detect_o : out std_logic
  );
end entity seq_detect;

architecture rtl of seq_detect is
  -- Each state names the longest matched prefix of 1011
  type t_state is (DET_IDLE, DET_1, DET_10, DET_101, DET_1011);

  signal state      : t_state;
  signal next_state : t_state;
begin
  reg_proc: process(rst_i, clk_i) is
  begin
```

```
    if rst_i = '1' then
      state <= DET_IDLE;
    elsif rising_edge(clk_i) then
      state <= next_state;
    end if;
  end process reg_proc;

  fsm_logic: process(state, bit_i) is
  begin
    next_state <= state;
    detect_o <= '0';
    case state is
      when DET_IDLE =>
        if bit_i = '1' then
          next_state <= DET_1;
        end if;
      when DET_1 =>
        if bit_i = '0' then
          next_state <= DET_10;
        end if;
      when DET_10 =>
        if bit_i = '1' then
          next_state <= DET_101;
        else
          next_state <= DET_IDLE;
        end if;
      when DET_101 =>
        if bit_i = '1' then
          next_state <= DET_1011;
        else
          next_state <= DET_10;
        end if;
      when DET_1011 =>
        detect_o <= '1';
        if bit_i = '1' then
          next_state <= DET_1;
        else
          next_state <= DET_10;
        end if;
    end case;
  end process fsm_logic;
end architecture rtl;
```

1. Draw the state diagram.

2. Translate the FSM into a SystemVerilog module `seq_detect` in `seq_detect/rtl/seq_detect.sv`, with an asynchronous active-low reset `rst_ni` instead of `rst_i` and otherwise the same port names, which the testbench below uses. Lint it.
3. Verify it with the following cocotb testbench, which compares `detect_o` with a Python reference model for a directed and a random bit stream. Run it with Icarus Verilog and with Verilator.

```
# SPDX-FileCopyrightText: 2026 Simon Dorrer
# SPDX-License-Identifier: Apache-2.0
# Description: Self-checking cocotb testbench for seq_detect with a
# Python reference model.

import os
import random
from pathlib import Path

import cocotb
from cocotb.clock import Clock
from cocotb.triggers import ClockCycles, FallingEdge
from cocotb_tools.runner import get_results, get_runner

PATTERN = [1, 0, 1, 1]


async def start_up(dut):
    """Start a 50 MHz clock and reset the DUT for two cycles."""
    cocotb.start_soon(Clock(dut.clk_i, 20, "ns").start())
    dut.rst_ni.value = 0
    dut.bit_i.value = 0
    await ClockCycles(dut.clk_i, 2)
    await FallingEdge(dut.clk_i)
    dut.rst_ni.value = 1


async def send_and_check(dut, bits):
    """Send one bit per cycle, compare detect_o with the model."""
    history = []
    detections = 0
    for bit in bits:
        dut.bit_i.value = bit
        # The DUT samples the bit at the rising edge in between
        await FallingEdge(dut.clk_i)
        history.append(bit)
        expected = 1 if history[-len(PATTERN):] == PATTERN else 0
        assert dut.detect_o.value == expected, \
            f"after bits ...{history[-8:]}: " \
```

```
            f"expected {expected}, got {dut.detect_o.value}"
        detections += expected
    return detections


@cocotb.test()
async def test_overlapping(dut):
    """1011011 contains the pattern twice, the second overlapping."""
    await start_up(dut)
    detections = await send_and_check(dut, [1, 0, 1, 1, 0, 1, 1, 0, 0])
    assert detections == 2, f"expected 2 detections, got {detections}"


@cocotb.test()
async def test_random_bits(dut):
    """A long random bit stream against the reference model."""
    await start_up(dut)
    rng = random.Random(1234)  # fixed seed: every run is reproducible
    bits = [rng.randint(0, 1) for _ in range(500)]
    detections = await send_and_check(dut, bits)
    # Guard against a test that passes without ever seeing the pattern
    assert detections > 0, "no pattern in the random stream"
    dut._log.info(f"{detections} detections in {len(bits)} bits")


def seq_detect_runner():
    sim = os.getenv("SIM", "icarus")
    proj_path = Path(__file__).resolve().parent

    runner = get_runner(sim)
    runner.build(
        sources=[proj_path / "../../rtl/seq_detect.sv"],
        hdl_toplevel="seq_detect",
        always=True,
        waves=True,
        timescale=("1ns", "1ps"),
    )
    results_xml = runner.test(
        hdl_toplevel="seq_detect",
        test_module="seq_detect_tb",
        waves=True,
    )

    # A failing test leaves the exit status at 0, so check the results
    num_tests, num_failed = get_results(results_xml)
```

```
    if num_failed:
        raise SystemExit(
            f"FAIL: {num_failed} of {num_tests} tests failed")


if __name__ == "__main__":
    seq_detect_runner()
```

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

Solution: Sequence Detector FSM

Each state stores the longest prefix of `1011` that the received bits end with. A `0` after `1011` leaves `10`, and a `1` leaves `1`, which is how overlapping occurrences are detected. Table 18 gives the state diagram as a table, with the state names of the VHDL version. In the SystemVerilog solution, `DET_IDLE` becomes `StIdle`, `DET_1` becomes `St1`, and so on.

Table 18: Transitions of the sequence detector

State	Next state for `bit_i = 0`	Next state for `bit_i = 1`	`detect_o`
`DET_IDLE`	`DET_IDLE`	`DET_1`	0
`DET_1`	`DET_10`	`DET_1`	0
`DET_10`	`DET_IDLE`	`DET_101`	0
`DET_101`	`DET_10`	`DET_1011`	0
`DET_1011`	`DET_10`	`DET_1`	1

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer
// SPDX-License-Identifier: Apache-2.0
// Description: Moore FSM that detects the bit pattern 1011 in the
// serial input bit_i (one bit per rising clock edge, overlapping
// occurrences count). detect_o is high for one cycle after the edge
// that sampled the last 1.

`default_nettype none

module seq_detect (
  input  logic clk_i,
  input  logic rst_ni,
  input  logic bit_i,
  output logic detect_o
);
```

```
// Each state names the longest matched prefix of 1011
typedef enum logic [2:0] {
  StIdle,
  St1,
  St10,
  St101,
  St1011
} state_e;

state_e state_d, state_q;

always_ff @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) begin
    state_q <= StIdle;
  end else begin
    state_q <= state_d;
  end
end

always_comb begin
  state_d  = state_q;
  detect_o = 1'b0;
  unique case (state_q)
    StIdle: begin
      if (bit_i) state_d = St1;
    end
    St1: begin
      if (!bit_i) state_d = St10;
    end
    St10: begin
      if (bit_i) state_d = St101;
      else       state_d = StIdle;
    end
    St101: begin
      if (bit_i) state_d = St1011;
      else       state_d = St10;
    end
    St1011: begin
      detect_o = 1'b1;
      if (bit_i) state_d = St1;
      else       state_d = St10;
    end
    default: state_d = StIdle;  // unused encodings
  endcase
```

```
  end

endmodule

`default_nettype wire
```

Five states need a 3-bit encoding, so three encodings are unused, and the `default` item sends them back to idle. Both simulators report `TESTS=2 PASS=2 FAIL=0 SKIP=0`, and the random stream of 500 bits contains the pattern 31 times. The testbench checks that this count is not zero, so that the random test cannot pass without ever seeing the pattern. Icarus Verilog prints a "sorry" message that it ignores `unique`, which you can disregard. Since cocotb is independent of the HDL, the same testbench also runs the VHDL version with GHDL (with the active-high reset `rst_i` instead of `rst_ni`), and both versions pass.

2.8 Hands-On: The Counter of the AMS Chip Template

In this course, you use the SG13CMOS5L version of the AMS chip template, ihp-sg13cmos5l-ams-chip-template, together with its tutorial. Its digital example is the macro `macros/counter`: an 8-bit counter with a synchronous reset, an enable, and a wrap-around at `CTR_MAX`, plus a wrapper `counter_top`, which converts the active-low reset of the chip into the active-high reset of the counter. The tutorial shows the complete code of the counter in its section `counter.sv`. The macro comes with a SystemVerilog and a cocotb testbench, and its `Makefile` wraps the commands of Section 2.7 into make targets. In the following exercises, you first type the commands yourself and only then look at the targets.

Exercise: Lint and Simulate the Counter by Hand

Clone the template into your designs folder (if you have not done so yet) and change into the counter macro:

```
cd /foss/designs
git clone https://github.com/iic-jku/ihp-sg13cmos5l-ams-chip-template.git
cd ihp-sg13cmos5l-ams-chip-template/macros/counter
```

1. Read `rtl/constants.sv`, `rtl/counter.sv`, and `rtl/counter_top.sv`.
2. Lint `counter_top` together with the files it needs with `verilator --lint-only -Wall`. In which order do you have to pass the files, and why?
3. Compile the SystemVerilog testbench `testbenches/verilog/counter_top_tb.sv` with Icarus Verilog, run it in its folder, and open the waveform with the saved signal list `counter_top_tb.gtkw`.
4. Run the cocotb testbench `testbenches/cocotb/counter_top_tb.py` with the default simulator and open its waveform.

5. The counter has four behaviors: reset, hold while disabled, increment while enabled, and wrap-around at `CTR_MAX`. Which of them does each testbench actually check?

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

Solution: Lint and Simulate the Counter by Hand

```
# Lint (no warning, even with -Wall)
verilator --lint-only -Wall rtl/constants.sv rtl/counter.sv rtl/
counter_top.sv

# SystemVerilog testbench with Icarus Verilog
iverilog -g2012 -o testbenches/verilog/counter_top_tb.vvp rtl/
constants.sv rtl/counter.sv rtl/counter_top.sv testbenches/verilog/
counter_top_tb.sv
cd testbenches/verilog
vvp counter_top_tb.vvp
gtkwave counter_top_tb.vcd counter_top_tb.gtkw

# cocotb testbench
cd ../cocotb
python3 counter_top_tb.py
gtkwave sim_build/counter_top.fst counter_top_tb.gtkw
```

- `constants.sv` defines the macros `` `COUNTER_MAX_DEFAULT `` and `` `CLK_FREQ_DEFAULT ``, which `counter_top.sv` and the testbench use. A macro must be defined before it is used, and the tools read the files in the given order, so `constants.sv` comes first.
- The SystemVerilog testbench prints `PASS: simulation complete.`, and the cocotb testbench reports `TESTS=4 PASS=4 FAIL=0 SKIP=0`.
- The cocotb testbench checks all four behaviors, one test each. The SystemVerilog testbench (template version of October 2026) checks only two: the counter is zero after the reset, and it stays zero while disabled. It enables the counter for a full wrap-around, but does not check the values, so increment and wrap-around are only visible in the waveform. A `PASS` only means that the implemented checks passed.

Exercise: The Counter in the Two-Process Style

1. Draw the counter as a synchronous sequential circuit in the structure of Figure 4. Which signal is the state, which are the inputs, and which operations does the next-state logic need?
2. Turn your diagram into the two-process style: rewrite `rtl/counter.sv` with `count_d` and `count_q`, the register in `always_ff` and the next-state logic in `always_comb`. Keep

the module name, the parameters, the ports, and the behavior (synchronous reset, enable, wrap-around at `CTR_MAX`) exactly as they are.

3. Lint the design and run both testbenches again. All checks must still pass, which shows that your refactoring did not change the checked behavior.
4. Now build in a bug: let the counter wrap one value early (compare with `CTR_MAX - 1`). Run both testbenches again and check their exit status with `echo $?`. Which testbench finds the bug? What does the exit status tell you?
5. Restore the original file with `git restore rtl/counter.sv`.

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

Solution: The Counter in the Two-Process Style

Figure 9 shows the counter in the structure of Figure 4. The state is `count_q`, held in `CTR_BW` flip-flops with the clock `clock_i` and the synchronous reset `reset_i`. The next-state logic gets `count_q` and the input `enable_i` and needs four parts: a comparator with `CTR_MAX`, an incrementer, a first multiplexer that selects zero at the wrap-around, and a second multiplexer that keeps `count_q` while `enable_i` is low.

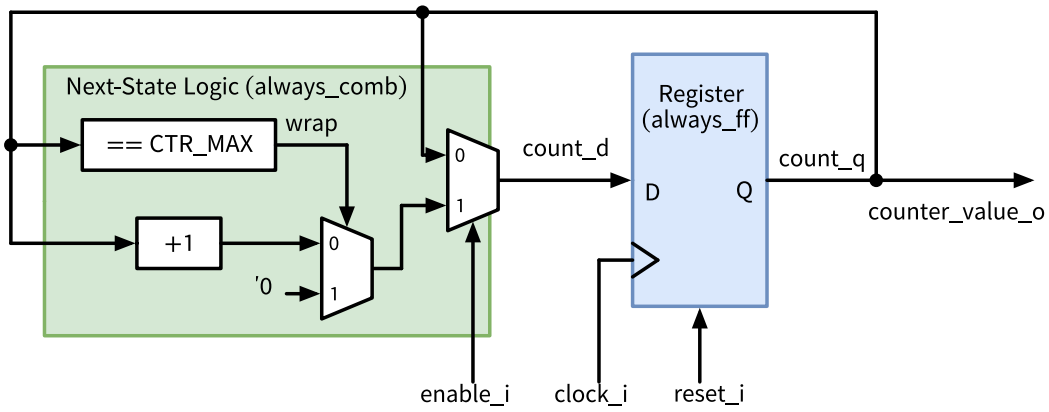


Figure 9: The counter of the AMS chip template as register and next-state logic, in the structure of Figure 4. The register holds `count_q` and has the synchronous reset `reset_i`. In the next-state logic, the comparator and the incrementer work on `count_q` in parallel, the first multiplexer selects zero at the wrap-around, and the second keeps `count_q` while `enable_i` is low.

In the code, the register becomes `always_ff`, and the next-state logic becomes `always_comb`, where the two multiplexers turn into the `if` and `else` decisions:

```
// SPDX-FileCopyrightText: 2026 Simon Dorrer and Harald Pretl
// SPDX-License-Identifier: Apache-2.0 WITH SHL-2.1
// Description: This file implements an N-bit up counter with
// synchronous reset & enable in SystemVerilog (two-process style).
```

```
`default_nettype none
`ifndef __COUNTER__
`define __COUNTER__

module counter #(
  parameter int unsigned       CTR_BW  = 8,
  parameter logic [CTR_BW-1:0] CTR_MAX = 2**CTR_BW - 1
)(
  input logic               clock_i,
  input logic               reset_i,
  input logic               enable_i,

  output logic [CTR_BW-1:0] counter_value_o
);

  logic [CTR_BW-1:0] count_d, count_q;

  // Register process: synchronous reset and copy of count_d only
  always_ff @(posedge clock_i) begin
    if (reset_i) begin
      count_q <= '0;
    end else begin
      count_q <= count_d;
    end
  end

  // Combinational process: next counter value, wraps around at CTR_MAX
  always_comb begin
    count_d = count_q;
    if (enable_i) begin
      if (count_q == CTR_MAX) begin
        count_d = '0;
      end else begin
        count_d = count_q + 1'b1;
      end
    end
  end

  assign counter_value_o = count_q;

endmodule // counter

`endif
`default_nettype wire
```

The synchronous reset stays in the register process, so `always_ff` holds the reset value and the copy of `count_d`, as in your VHDL `reg_proc`. Putting the reset into the combinational process as the first decision would describe the same hardware. Lint and both testbenches pass as before.

With the bug, the SystemVerilog testbench still prints `PASS` and exits with status 0, because it does not check the wrap-around. In the cocotb testbench, `test_wraps_at_max` fails, but `python3 counter_top_tb.py` still exits with status 0, because the runner of the template does not evaluate `results.xml` (compare it with the runner of Section 2.6.2). Two lessons follow: a testbench is only as good as its checks, and an exit status is only useful if the script sets it. Both matter as soon as testbenches run automatically, e.g., in a regression with `make` or in continuous integration.

Exercise: What Is Behind the Make Targets?

Open `macros/counter/Makefile` of the template.

1. `make help` lists all targets. Try it, then try `make -n sim-rtl-verilog`, which prints the commands of a target without running them (dry run). What does the Makefile check before it runs a target, and what do you have to do if this check fails?
2. Find the targets that run the commands you typed in the previous exercises, and compare their commands (`make -n <target>`) with yours. What is different?

Before continuing, please finish the previous exercise. Once you are done, compare with the solution below.

Solution: What Is Behind the Make Targets?

Every target except `help` and `clean` first checks that the environment variable `PDK` is `ihp-sg13cmos5l`, because the macro is built for this PDK. If the shell is set to another PDK (the default of the container is `ihp-sg13g2`, check it with `echo $PDK`), the Makefile stops with an error message, even for a dry run. Switch the PDK with `sak-pdk ihp-sg13cmos5l` in the same shell and try again. Your own commands did not need this, because an RTL simulation does not use the PDK. Table 19 compares the targets with your commands.

Table 19: The make targets of the counter macro and the commands behind them

Your command	Make target	Differences
`verilator --lint-only -Wall ...`	`make lint-verilog`, `make lint-verilog CELL=counter`, `make lint-verilog-all` (both)	without `-Wall`, with the include path `-I"rtl"`, `CELL` selects the files
`iverilog -g2012 ...` and `vvp ...`	`make sim-rtl-verilog`	adds `-DSIM`, names the output `test_<CELL>` and deletes it after the run
`gtkwave counter_top_tb.vcd counter_top_tb.gtkw`	`make sim-view-verilog`	`WAVEFORM_VIEWER=surfer` opens Surfer with its save file `.surf.ron`
`python3 counter_top_tb.py`	`make sim-rtl-cocotb`	the same command, `make sim-gl-cocotb` sets `GL=1` for the gate-level netlist
`gtkwave sim_build/counter_top.fst ...`	`make sim-view-cocotb`	as `sim-view-verilog`

After the switch, `make -n sim-rtl-verilog` prints:

```
iverilog -g2012 -DSIM -o testbenches/verilog/test_counter_top rtl/
constants.sv rtl/counter.sv rtl/counter_top.sv testbenches/verilog/
counter_top_tb.sv
cd testbenches/verilog && vvp test_counter_top
rm -f testbenches/verilog/test_counter_top
```

The variable `CELL` (default `counter_top`) selects the cell, e.g., `make lint-verilog CELL=counter`. From now on, you can use the make targets, knowing what they do.

2.9 HDL Coding Guidelines

> "A consistent naming and schematic drawing style, as well as VHDL/Verilog coding scheme, is a huge help in avoiding errors and increasing productivity." (Circuit Designer's Etiquette)

The VHDL/Verilog Coding Guide of the Circuit Designer's Etiquette was written for Verilog and applies to SystemVerilog with small adaptations. Its rules, restated for the SystemVerilog of this course:

1. **Lint every file** with `verilator --lint-only -Wall` and fix every warning. A warning you ignore today hides the one that matters tomorrow.

2. **Write readable code** with descriptive names and comments that explain the intention. Inputs end in `_i`, outputs in `_o`, bidirectional ports in `_io`. An active-low signal has an `n` before the direction suffix, e.g., `rst_ni`.
3. **Choose the reset deliberately.** A synchronous reset avoids reset races. An asynchronous reset, our default, must be free of glitches and released synchronously (Section 9).
4. **Use `logic` for all signals.** The suffixes `_w` and `_r` of plain Verilog for `wire` and `reg` are not needed.
5. **Use `assign` for simple logic.** The conditional operator `c ? a : b` is often clearer than a nested `if`.
6. **Declare every port explicitly** with direction, type, and width (`output logic [7:0] data_o`).
7. **Use `localparam` for constants** and write parameters in UPPERCASE.
8. **Reset every register that holds control state**, e.g., FSM states, counters, and valid flags, to a defined value.
9. **One module per file**, and the file name equals the module name.
10. **Start every file with `` `default_nettype none ``** and end it with `` `default_nettype wire ``, as a safety net against typos.
11. **Use `always_comb`** for combinational logic. It replaces the `always @(*)` of plain Verilog and never has an incomplete sensitivity list.
12. **Parameterize** bit widths and other values with parameters and `localparam`s.
13. **Make conversions and widths explicit** instead of relying on implicit extension and truncation (Section 2.5.3).
14. **Use blocking assignments `=` only in `always_comb` and non-blocking assignments `<=` only in `always_ff`.**
15. **Make visible whether a control acts synchronously or asynchronously.** The etiquette names synchronous set and reset `set` and `res`, and asynchronous ones `pre` (preset) and `clr` (clear). In this course, the asynchronous reset of a block is `rst_ni`, and a synchronous reset or clear is an ordinary input, such as `reset_i` in the counter of the AMS chip template.

From the lowRISC style guide [7], we additionally adopt the suffixes `_d` and `_q` for the next and current value of a register, `_e` for enumeration types and `_t` for other types, UpperCamelCase state names, `unique case` with a `default`, named connections for every port and parameter, and no plain `always` blocks. For parameters, however, we keep the UPPERCASE names of the etiquette (rule 7), while lowRISC uses UpperCamelCase. Finally, Table 20 collects the traps that VHDL designers fall into most often.

Table 20: Pitfalls for VHDL designers

Pitfall	What happens	Fix
Expecting `reg` to be a register	`reg` and `logic` are types, storage follows from the block	use `logic`, look at the block
Blocking `=` in `always_ff`	order-dependent results, simulation and synthesis differ	`<=` in `always_ff`
Missing `else` or `default` in `always_comb`	a latch	default assignments first
Case-insensitive habits from VHDL	`Clk` and `clk` are two signals	lowercase snake_case
A typo in a signal name	an implicit 1-bit wire, silently unconnected	`` `default_nettype none ``, lint
Mismatched widths	silent truncation or extension, e.g., a lost carry	sized literals, explicit widths, lint
Mixing signed and unsigned	the whole expression is unsigned	`$signed()`, consistent types
`==` with `x` in a testbench check	the result can be `x`, the check passes silently	`!==` or `$isunknown()` in testbenches
`?:` between enumeration values	Icarus Verilog requires an explicit cast	`if` and `else`
`int` for counters in RTL	32 bits, two-state, hides `x`	`logic [N-1:0]`
`initial` for reset values	not synthesizable for an ASIC	a reset in `always_ff`
Testbench drives on the active clock edge	races, results differ between simulators	drive and check on the falling edge
`$error` in an Icarus Verilog testbench	exit status 0 despite the failure	`$fatal(1, ...)`

2.10 Summary

- An HDL describes hardware that works concurrently, not instructions that run one after another. Think hardware first, then write the code.
- Almost every digital circuit is a register plus combinational next-state logic. The register loads the next state at every rising clock edge, so the logic has one clock period to compute it.
- Many HDLs exist, but all alternatives can generate (System)Verilog, which then runs through the same tools. SystemVerilog is the industry standard and the language of our open-source flow.
- Your VHDL two-process style maps directly onto SystemVerilog: `always_ff` holds the reset values and copies `_d` into `_q`, and `always_comb` starts with defaults and contains all decisions.
- Use `logic`, `always_comb` with `=`, `always_ff` with `<=`, sized literals, and an asynchronous active-low reset `rst_ni`. Lint every file and check it with a quick Yosys synthesis.

- Plain Verilog is everywhere, in netlists, cell models, generated code, and older IP. Read it, but write SystemVerilog: `logic` instead of `reg` and `wire`, `always_comb` and `always_ff` instead of `always`.
- Make every testbench self-checking, drive and check on the falling clock edge, and make sure that a failure sets the exit status: `$fatal` in SystemVerilog, `get_results()` in a cocotb runner.
- The make targets of the AMS chip template wrap the commands you typed in this chapter, with small differences, e.g., `-DSIM` and no `-Wall` (Table 19).

In Section 3, we look at modeling guidelines that turn correct RTL into RTL that synthesizes well, and Section 4 takes the cocotb testbenches of this chapter to a systematic verification methodology.

3 Modeling Guidelines for Synthesizable RTL

> **Note**
>
> This chapter will be released later in the semester.

4 Functional Verification with cocotb

> **Note**
>
> This chapter will be released later in the semester.

5 Sequential Elements and Static Timing Analysis

> **Note**
>
> This chapter will be released later in the semester.

6 The Synthesis Library

> **Note**
>
> This chapter will be released later in the semester.

7 Synthesis Optimizations

> **Note**
>
> This chapter will be released later in the semester.

8 Design for Test (DFT)

> **Note**
>
> This chapter will be released later in the semester.

9 Clock Domains and Metastability

> **Note**
>
> This chapter will be released later in the semester.

10 The RTL-to-GDSII Flow with LibreLane

> **Note**
>
> This chapter will be released later in the semester.

11 Capstone Project: A Small RISC-V SoC

> **Note**
>
> This chapter will be released later in the semester.

12 Appendix: SystemVerilog Cheat-Sheet

> **Note**
>
> This appendix will be released later in the semester.

13 Appendix: Tooling Cheat-Sheet

> **Note**
>
> This appendix will be released later in the semester.

14 Appendix: Sample Exam Questions

> **Note**
>
> This appendix will be released later in the semester.

Bibliography

[1] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021.

[2] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Addison-Wesley, 2010.

[3] C. E. Cummings, *Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog*. Sunburst Design, 2008.

[4] Institute for Integrated Circuits, JKU, "IIC-OSIC-TOOLS: A Container with Open-Source IC Design Tools."

[5] S. Dorrer and H. Pretl, "An Open-Source Analog Mixed-Signal Chip Design Template & Tutorial for the ihp-sg13cmos5l Open-PDK." 2026. doi: 10.5281/zenodo.22115134.

[6] IEEE, "IEEE Standard for SystemVerilog: Unified Hardware Design, Specification, and Verification Language." 2024. doi: 10.1109/IEEESTD.2024.10458102.

[7] lowRISC, "lowRISC Verilog Coding Style Guide."

[8] S. Sutherland, *RTL Modeling with SystemVerilog for Simulation and Synthesis: Using SystemVerilog for ASIC and FPGA Design*. Sutherland HDL, 2017.

[9] S. Sutherland and D. Mills, *Verilog and SystemVerilog Gotchas: 101 Common Coding Errors and How to Avoid Them*. Springer, 2007. doi: 10.1007/978-0-387-71715-9.

[10] cocotb developers, "cocotb: A Coroutine-Based Cosimulation Testbench Environment for Verifying VHDL and SystemVerilog RTL Using Python."